



MASTER THESIS REPORT

Quantitative Security Evaluation of Protocols: from Symbolic Verification towards Timed Models

Nicolas Matei

Master's in Mathematics and Applications – IT Mathematics and Cryptography

Supervisors:

Benjamin Bispin
Natalia Kushik
Philipp Schlehuber-Caissier

Host Institution:

Télécom SudParis, Institut Mines-Télécom
Laboratoire SAMOVAR
RS2M department

Defense on September 3, 2026

Acknowledgements

I would like to express my sincere gratitude to my three supervisors, Benjamin Bispin, Natalia Kushik, and Philipp Schlehuber-Caissier, for their invaluable guidance and for the opportunity to complete such a rewarding internship. I would also like to extend a special thanks to Evgenii Vinarskii, Jakub Hajdas, and Hanzhe Yang.

I am grateful to David Baelde, JosephALLEMAND, and Thomas Genet for my 2025 internship, where I was first introduced to the field of formal methods.

Thanks to Felix Ulmer, Reynald Lercier, and Luc Aberger for teaching the only things I ever really understood about abstract mathematics. I also extend my thanks to Mathieu Goessens, an outstanding teacher who made Networking engaging, and to Sylvain Duquesne for introducing me to the world of cryptography.

Contents

1	Introduction	4
1.1	Motivation, Context, and Problem Statement	4
1.2	Contributions	5
2	Presentation of the Lab and the Team	6
2.1	Presentation of the Research Lab	6
2.2	Presentation of Tutors and research colleagues from the institute	6
2.3	Presentation of Orange	7
2.4	My Role	7
3	Preliminaries and Related Works	8
3.1	Related work (Exterior to the team)	8
3.2	Preliminaries (Exterior to the team)	8
3.2.1	Modeling and Verification in ProVerif	8
3.2.2	ProVerif's Attack Reconstruction Algorithm	9
3.2.3	Toy example of a .pv file	10
3.2.4	parser <code>extract_pi.py</code>	10
3.2.5	Set of Assumptions	12
3.2.6	Timed automata	12
3.3	Preliminaries (within the team - CompareVerif)	13
3.3.1	Tooling to Survey Attacks and Adversary Capabilities in ProVerif	13
3.3.2	Checking Protocol Security	16
4	Contributions to CompareVerif	17
4.1	Historical Context	17
4.2	Communication at AFADL GDR Scilog 2026 and at ACMES Seminar at TSP	17
4.3	Contributing to the Conference Paper for SEFM 2026	18
4.4	Reasoning with Capabilities	18
4.4.1	Mathematical Framework for Capabilities	18
4.4.2	Analyzing a simple ratchet protocol	19
4.5	Learning from the experience with CompareVerif	19
5	Experimenting with a randomization feature on ProVerif's algorithm	21
5.1	Modifying ProVerif's Attack Reconstruction Algorithm	21
5.2	Simple Example	21
5.3	Possible Improvements to the modified algorithm of Proverif	22
6	Improving the Tree Generation of ProVerif	23
6.1	Context and Motivation	23
6.2	Hypotheses	23
6.3	Definitions	24
6.4	Specification and Implementation of the generation of attack trees	24
6.4.1	Specification	24
6.4.2	Implementation	26

6.4.3	Results	26
6.5	Further transforming the directory of Conjunctive Trees into a Master Forest	29
6.5.1	Specification	29
6.5.2	Implementation	29
6.5.3	Results	29
6.6	Comparing the new Tree Generation with CompareVerif's Tree generation	32
7	Towards Timed Proverif	33
7.1	Assumptions and Metrics	35
7.1.1	Modeling assumptions	35
7.1.2	Extraction of atomic actions and legend	35
7.1.3	Modeling the protocol's execution tree	35
7.1.4	Generation of attack metrics	36
7.1.5	Global variables and inter-component dependencies	38
7.2	Generation of the automaton	38
7.2.1	Generation of the protocol automaton	38
7.2.2	Generation of the attack automaton	41
7.2.3	Generation of the property automaton	42
7.2.4	Generation of final automaton	44
7.3	Implementation	44
7.3.1	Trying out the implementation	44
7.3.2	Experimenting with the implementation	44
7.4	Future Improvements	44
7.4.1	Implementing the tool into a sound, functional, and user-friendly artefact	44
7.4.2	Outlining how to solve the combinatorial state explosion system	44
8	Conclusions	46
8.1	Results	46
8.2	Future Works	46
8.2.1	Related to CompareVerif	46
8.2.2	Related to Randomized Proverif	46
8.2.3	Related to the Attack Tree Generation	47
8.2.4	Related to Timed Proverif	47
8.2.5	Abstractions for EFSMs	47
	References	50
	Appendix - SEFM Conference Paper	51

Chapter 1

Introduction

1.1 Motivation, Context, and Problem Statement

Formal methods and symbolic verification are widely used to evaluate the security properties of communication systems. However, traditional symbolic protocol verifiers, such as ProVerif [5] and Tamarin [16], typically offer a strictly binary assessment: they determine whether a property holds, yielding simple *Yes* or *No* answers alongside a potential attack trace, but lack quantitative depth. This distinction is critical: a security property might be theoretically violated, but if the cost of executing the attack is prohibitively high, the system remains practically secure. Consequently, a fundamental question drives our endeavor: if a vulnerability exists, how much effort, time, or energy must an attacker expend to exploit it? Quantifying the cost of an attack trace grounds these theoretical tools in engineering reality, making formal methods more applicable to real-world situations.

Shifting to this quantitative perspective naturally introduces a broader challenge in security engineering: benchmarking the resilience of communication systems. Rather than simply asking if a configuration is secure in an absolute sense, the goal is to establish a rigorous baseline of its vulnerabilities. While theoretical perspectives often focus on an attacker’s computational limits, the engineering perspective prioritizes system resilience, asking: how many components must fail, or what is the precise accumulated cost, for the system to be compromised?

Traditionally, this accumulated cost is evaluated using attack trees [22] and vulnerability comparisons [17]. When augmented with security metrics [19], attack trees can assign a total price to an attack scenario, providing a quantitative security assessment against a given threat model. Deriving comprehensive attack trees manually is notoriously difficult; leveraging automated security checking tools offers a highly promising solution to automatically generate and price these scenarios.

Unfortunately, existing symbolic provers are not natively designed for this comparative, quantitative approach. They typically analyze an *individual* system under a single, fixed threat scenario. Currently, the state of the art relies on manual, ad-hoc methods—such as commenting and uncommenting parts of a model to test variants, or using external preprocessors to enumerate configurations. ProVerif is then executed, and the resulting data is aggregated manually to interpret the required attacker capabilities [8, 11, 21]. This fails to integrate varying attacker capabilities explicitly into the verification process, leaving a disconnect between formal proofs and quantitative frameworks like attack tree pricing.

Furthermore, meaningful security comparisons often require evaluating dynamic defensive mechanisms, which relies on concurrently modeling offensive actions and a system’s temporal behavior. A prime example is Moving Target Defense (MTD), which dynamically shifts the attack surface to render attack traces prohibitively expensive. Symbolic provers abstract away the precise temporal/timed and state-dependent behaviors essential for analyzing such time-sensitive mechanisms. Timed model checkers, such as UPPAAL, are specifically designed to capture these exact temporal dynamics; however, they traditionally lack the robust cryptographic abstractions provided by symbolic verification workflows.

The goal of this research is to bridge this gap. We propose a general methodology and an integrated toolset to quantitatively assess a protocol’s resilience by combining the logical abstractions of symbolic verifiers with the temporal precision of timed model checkers.

1.2 Contributions

When I arrived at the Institute, Benjamin Bisping already had a first prototype of the tool CompareVerif (<https://github.com/benkeks/compareverif>), described in [subsection 3.3.1](#). CompareVerif enumerates attack scenarios specified in a single file and uses the ProVerif verifier to automatically measure protocol security in a quantitative fashion. It also enhances the convenience of security benchmarking by providing augmented attack graphs and Pareto fronts of attack budgets. CompareVerif slightly generalizes the Dolev-Yao attacker model used by ProVerif.

Contributions to already existing CompareVerif: I contributed to Benjamin Bisping’s tool, originally built a few weeks before my arrival at the institute. I helped with exhaustive debugging, and helped write the first documentation and formal definition of the algorithm and implementation described in [section 4.4](#). This documentation would be further improved by everyone in the team to lead to the submission of a Conference Paper at SEFM 2026, currently under review, with CompareVerif as artefact submission. I helped in finding and documenting use cases of the artefact. This is further explained in [subsection 4.4.2](#).

Conference Presentation at AFADL, GdR SciLog 2026: I helped with exhaustive communication about CompareVerif and the project with Orange. Natalia Kushik and I presented a talk at a conference and at a laboratory seminar. This is further explained in [section 4.2](#).

Randomized ProVerif: Whenever ProVerif finds an attack, it will stop looking for other attacks. In order to have more attack traces, whether they be attack trees or message sequence charts (MSCs), we slightly change ProVerif’s algorithm to find other attacks, with the altered implementation provided at https://github.com/NicolasMatei/randomized_proverif. This is further explained in [chapter 5](#). We explain the changes to the algorithm, and give an example of application.

New Generation of Attack Trees from ProVerif: CompareVerif, along with its main feature of automation of benchmarking, comes with a generation of Attack Trees. However this generation of attack trees has room for improvement. We specify a new algorithm for the generation of attack trees (available at https://github.com/NicolasMatei/Attack_Tree_Generation) and compare it with the attack tree generation of CompareVerif in [section 6.6](#). Optionally this tool can use Randomized ProVerif in order to have a more exhaustive generation of Attack Trees.

Pipelining ProVerif to UPPAAL: We present a framework to pipeline the Generated Attack Trees from before into Timed automaton in [chapter 7](#). This is a work in progress.

Chapter 2

Presentation of the Lab and the Team

2.1 Presentation of the Research Lab

This internship took place at **Télécom SudParis**, a graduate engineering school located on the Évry-Courcouronnes campus (9 rue Charles Fourier, 91011). It is part of the **Institut Mines-Télécom (IMT)**, the leading public group of engineering and management schools in France. The internship was held on the secondary campus at Place Marguerite Perey in Palaiseau.

The research was conducted within the **SAMOVAR** (Distributed Services, Architectures, Modeling, Validation, Network Administration) laboratory, which is hosted directly on the school's premises. It specializes in information and communication sciences, notably future networks, cybersecurity, and artificial intelligence.

More specifically, I joined the **RS2M** (Networks, Systems, Services, Models) department within this laboratory. Its expertise focuses on the evolution of digital infrastructures: architecture design, performance evaluation, virtualization (SDN/NFV), and the Internet of Things (IoT).

2.2 Presentation of Tutors and research colleagues from the institute

Benjamin Bisping Postdoctoral researcher in cybersecurity at Télécom SudParis (Institut Polytechnique de Paris). His research mainly focuses on the semantics and security of distributed systems, concurrency theory, as well as the analysis of behavioral equivalences through quantitative games and formal methods.

Natalia Kushik Associate Professor at Télécom SudParis within the ACMES team of the SAMOVAR laboratory. An expert in formal methods, her research targets model-based testing for network security. Furthermore, she studies indistinguishability for numerous families of finite automata.

Philipp Schlehuber-Caissier Associate Professor at Télécom SudParis, researcher within the SAMOVAR laboratory. His areas of expertise intersect formal methods, software verification, and control theory. He is particularly interested in timed automata theory, reactive controller synthesis, and the UPPAAL model checking tool.

Evgenii Vinarskii PhD student in computer science at Télécom SudParis (within the SAMOVAR laboratory). His research focuses on timed automata theory and model-based testing for network services. Having joined the Orange project at the end of July, he has since dedicated his activity to the applications of his research.

Research Environment All of the research colleagues in Computer Science meet on the fourth floor. The laboratory is a very lively place, with interaction between researchers. I had the opportunity to discuss with PhD students working in different fields.

2.3 Presentation of Orange

Orange is one of the world's leading telecommunications operators and the leader in France. The group offers comprehensive connectivity services including mobile telephony (4G/5G), very high-speed internet access (optical fiber), and digital services for individuals. Beyond the general public, its Orange Business division supports the digital transformation of companies and institutions with advanced solutions in networks, data integration, cloud computing, and the Internet of Things (IoT). Furthermore, the company establishes itself as a major European player in cybersecurity through Orange Cyberdefense. Present internationally, notably with a strong footprint in Europe and Africa, Orange connects hundreds of millions of users daily while placing digital inclusion and the ecological transition at the heart of its strategy.

Orange was a research partner for the research project. This collaboration was conducted with **Jean-Philippe Wary** (expert and director of cybersecurity research projects, specializing in 5G/6G mobile network security) and **Chrystel Gaber** (researcher specializing in the formal evaluation of security and privacy risks at Orange Labs). Our research team had monthly meetings with them to discuss the applications of formal methods in a Telecom context. Our problem formulation resulting from these meetings was stated at the AFADL GdR SciLog 2026 conference [15]. The research paper currently being submitted to SEFM 2026 addresses one of these problems stated in the conference presentation at AFADL GdR SciLog 2026.

2.4 My Role

During my internship, I joined a team of three researchers. My first role was to learn: how to collaborate, conduct research, communicate with the community, and write papers. My second focus was to actively participate in meetings, where I produced new ideas and gave suggestions. Since this was a research-oriented internship in computer science, I was involved in both fundamental studies and R&D. Notably, I participated in developing algorithms and tools to efficiently use formal methods in systems' security.

Chapter 3

Preliminaries and Related Works

3.1 Related work (Exterior to the team)

This thesis builds upon existing work by combining pricing with automated symbolic verification to systematically measure and benchmark the security levels of communication protocols. Additionally, it extends this quantitative foundation by integrating timed analysis into these symbolic evaluations, allowing for the evaluation of both time and price.

Attack scenarios in ProVerif. Jacomme and Kremer [11], model and enumerate threat scenarios in ProVerif to compare the security of multi-factor authentication mechanisms. The approach is continued by Paier et al. [21], where threats are described through interfaces of protocol entities and their level of access to them. Scenarios are enumerated by a preprocessing tool, which is partially supported by ProVerif [7], but not by the IDE [18].

Security benchmarking. A key aspect of the work addresses how to benchmark the security of communicating systems, a natural question in the software engineering community. For example, Oliveira et al. [20] turn to a trustworthiness score metric. We bring similar reasoning to the symbolic verification context.

Quantitative attack trees. In model-based security analysis, it is common to learn more about the feasibility of implementing attacks by deriving a quantitative conclusion from symbolic structures. For example, Schneier [22] already augments attack trees with prices such that the total price of the attack is computed from the prices for relevant branches, taking into account the type of the tree (e.g., AND or OR attack structure). Kumar et al. [12] perform a more refined quantitative analysis of attack trees, generalized to a logic for expressing various metrics on attack trees by Nicoletti et al. [19].

Moving Target Defense and Timed Analysis. Dynamic mitigation strategies, such as Moving Target Defense (MTD), introduce temporal constraints and cost trade-offs between security and quality of service. Evaluating these mechanisms quantitatively requires reasoning about multi-step attack sequences and timing. For instance, Ballot et al. [2] introduce a DAG-based formalism to model MTDs, translating it into a Price Timed Markov Decision Process. They utilize UPPAAL Stratego to compute an attacker's time- and cost-optimal strategies against given MTD activation frequencies, allowing defenders to subsequently evaluate and optimize their configurations.

3.2 Preliminaries (Exterior to the team)

3.2.1 Modeling and Verification in ProVerif

Symbolic protocol verifiers evaluate security properties by modeling protocols alongside an algebraic theory of message transformation. In this work, we use ProVerif [5], an automated symbolic verification tool that models protocols using the applied pi-calculus (π -calculus). ProVerif operates under the

baseline reference of the Dolev-Yao attacker model, which assumes the adversary has full control over all exposed communication channels. In practice, this means the attacker is capable of intercepting, reading, blocking, deleting, injecting, modifying, and replaying any message transmitted over the network. This approach excels at uncovering vulnerabilities such as man-in-the-middle attacks (e.g., on the Needham-Schroeder protocol [14]). The only limitation of this attacker is the inability to break cryptographic primitives without possessing the appropriate keys (which are modeled through equations). Note that probabilistic and asymptotic cryptographic properties, such as computational indistinguishability, are inherently abstracted away.

The applied pi-calculus used to model these systems is defined by a specific grammar encompassing terms (variables, names, constructors), conditions (equalities, logical operators), and processes. This language is enriched with practical features for modeling real-world systems, such as tables to represent databases, communication channels (**in**, **out**), and events to mark execution steps.

Crucially, it is necessary to distinguish between structural constructs and the *atomic* actions of the applied pi-calculus.

Formal Definition: Grammar of the applied Pi-Calculus: Terms and Structural Constructs

```
// --- TERMS (Messages and Data) ---
term, M, N ::=
  x, y, z                // Variables
  a, b, c, n             // Names (channels, nonces, keys...)
  f(M_1, ..., M_k)       // Constructors / destructors

// --- CONDITIONS ---
cond ::=
  M = N                  // Equality
  M != N                 // Inequality
  cond & \& cond         // Logical AND
  cond || cond           // Logical OR

// --- NON-ATOMIC PROCESSES (Structural Constructs) ---
P, Q ::=
  0                      // Null process (end)
  P | Q                  // Parallel composition
  !P                     // Replication
  new n: type; P          // Name creation (nonce, key)
```

Formal Definition: Subset of atomic actions in the applied Pi-Calculus

```
P, Q ::=
  in(M, x: type); P      // Message reception
  out(M, N); P           // Message emission
  if cond then P else Q  // Conditional
  let x = M in P else Q  // Assignment (pattern matching)
  event e(M_1, ..., M_k); P // Event triggering
```

It is paramount to emphasize the role of these specific **atomic actions**. In [chapter 7](#), these atomic actions will be precisely used to establish a one-to-one mapping (a bijection) between the execution steps of the protocol and some of the states of the timed automaton.

3.2.2 ProVerif's Attack Reconstruction Algorithm

To verify security properties, ProVerif over-approximates the protocol's behavior using a constraint system and a Horn clause derivation system in order to find potential attacks. The workflow begins with the user writing down the equational theory, the applied pi-calculus specification, and a queried

property into a .pv file. ProVerif then transforms this file into a set of Horn clauses and applies a resolution algorithm.

If the set of Horn clauses cannot be satisfied, then we know that the queried property is not violated. Conversely, if the set of Horn clauses is satisfiable, a Horn Derivation Proof Tree is generated. However, because the system uses an over-approximation, this Proof Tree does not necessarily represent a realistically executable attack on the concrete protocol. To address this, ProVerif applies its Attack Reconstruction Algorithm to determine if the Tree maps to a genuine attack trace. If the reconstruction is successful, ProVerif decides that the queried property is indeed violated; otherwise, it cannot make a definitive decision.

Unfortunately, whenever Proverif finds an attack, it will stop trying to look for other attacks. In that regard whenever a security breach in a protocol specification becomes a source of multiple attacks, Proverif might miss out on some of the most important attacks. We partially solve this problem in [chapter 5](#).

3.2.3 Toy example of a .pv file

For the purpose of this thesis we built a small .pv file in [figure 3.1](#) which will serve as a toy example throughout the document multiple times.

```

reduc forall m: bitstring, k: bitstring; dec(enc(m, k), k) = m.
reduc forall m: bitstring, k: bitstring; unhash(hash(m, k), k) = m.

query attacker(success).
query attacker(success2).

let C() =
  in(c,x:bitstring);
  if hash(nonce,seed) = x || enc(nonce,sk) = x then
    out(c,success).

let C2() =
  in(c,x:bitstring);
  in(c,y:bitstring);
  if hash(nonce2,seed) = x && enc(nonce3,sk) = y then
    out(c,success2).

let Oracle() = out(c,secret).

let O_enc() = in(c,x:bitstring);in(c,y:bitstring); if y = secret then out(c,enc(x,sk)).

let O_hash()= in(c,x:bitstring);in(c,y:bitstring); if y = secret then out(c,hash(x,seed)).

process
  C() || C2() || Oracle() || O_enc() || O_hash() || out(c,nonce) || out(c,nonce2) || out(c,
  nonce3)

```

Figure 3.1: Reduced content of a ProVerif system from `how_many_attack.pv` which serves as a toy example throughout the whole document.

3.2.4 parser `extract_pi.py`

Proverif comes with a native parser for the system. It assigns numbers we call pi-ids to atomic actions in the system. We use a little script `extract_pi.py` which calls this parser. This will be extensively used in [chapter 6](#) and [chapter 7](#).

```

(
  {1}in(c, x: bitstring);
  {2}if ((hash(nonce,seed) = x) || (enc(nonce,sk) = x)) then
  {3}out(c, success)
) | (
  {4}in(c, x_1: bitstring);
  {5}in(c, y: bitstring);
  {6}if ((hash(nonce2,seed) = x_1) && (enc(nonce3,sk) = y)) then
  {7}out(c, success2)
) | (
  {8}out(c, secretcondition)
) | (
  {9}in(c, x_2: bitstring);
  {10}in(c, y_1: bitstring);
  {11}if (y_1 = secretcondition) then
  {12}out(c, enc(x_2,sk))
) | (
  {13}in(c, x_3: bitstring);
  {14}in(c, y_2: bitstring);
  {15}if (y_2 = secretcondition) then
  {16}out(c, hash(x_3,seed))
) | (
  {17}out(c, nonce)
) | (
  {18}out(c, nonce2)
) | (
  {19}out(c, nonce3)
)

```

Figure 3.2: Parsing of the toy example from 3.1, by `extract_pi.py`. Notice how Proverif assigns numbers, we call pi-ids, to the atomic actions in the system. This will become relevant information for [chapter 6](#) and [chapter 7](#).

3.2.5 Set of Assumptions

Consider a system in Applied Pi-Calculus.

Twice in this document, we will need to call upon the following set of assumptions:

Modeling Assumptions:

Assumption 1: Absence of replication (bounded process)

The studied system does not make use of the replication operator (infinite loop), denoted $!P$ in the standard grammar of the applied pi-calculus.

Assumption 2: Parallelism restricted to the root (static processes)

The parallel composition operator ($P \mid Q$) is only allowed at protocol initialization (at the root, or *top-level*). No dynamic sub-process (after an *in*, *out* or *if* action) can spawn new parallel processes.

A Formal consequence is that the initial state of the system S is written canonically in the form

$$S = P_1 \mid P_2 \mid \cdots \mid P_n$$

where, for every $i \in \{1, \dots, n\}$, the process P_i is purely sequential (its grammar rule does not contain the symbol $\mid$).

3.2.6 Timed automata

We briefly recall the standard notion of a timed automaton [1] on which UPPAAL [13] is based. We mention that in this model, variables called "clocks" move towards the future.

Formal Definition: Extended Timed Automaton

An extended timed automaton is a tuple

$$\mathcal{A} = (L, l_0, C, Var, \Sigma, E, Inv)$$

where:

- L is a finite set of *locations* (the control states);
- $l_0 \in L$ is the initial location;
- C is a finite set of real-valued *clocks*;
- Var is a finite set of discrete *variables* (e.g., integers or booleans);
- Σ is a finite alphabet of actions (synchronization labels);
- E is the set of *transitions*, a transition $(l, g, a, r, u, l') \in E$ being read as "from l , if the guard g (a constraint over $C \cup Var$) is satisfied, the automaton may perform action a , reset the clocks in r , apply the variable update u to Var , and reach l' ";
- Inv associates to each location an *invariant* (a constraint over C), restricting the time the automaton may remain in that location.

The following definition formalizes the parallel composition of multiple extended timed automata, adopting the standard network semantics used by UPPAAL [3].

One could think of parallel composition in the category of timed automata as a tensor product, which equips the category with a symmetric monoidal structure. This intuitively means that we run multiple timed automata in parallel with the variables and the clocks put in common.

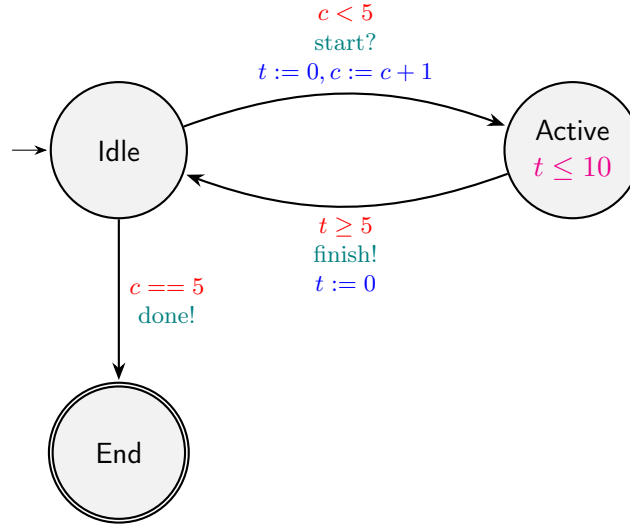


Figure 3.3: Model of an extended timed automaton. t is a continuous clock and c is a discrete integer variable. (In **red**: guards, in **magenta**: invariants, in **teal**: actions, in **blue**: updates/resets).

Formal Definition: Parallel Composition $\mathcal{A}_1 \otimes \dots \otimes \mathcal{A}_n$

Given n extended timed automata $\mathcal{A}_k = (L_k, l_0^k, C_k, Var_k, \Sigma_k, E_k, Inv_k)$, $k = 1, \dots, n$, with pairwise disjoint clocks ($C_k \cap C_{k'} = \emptyset$ for $k \neq k'$) but whose variable sets Var_k may intersect. We define the product

$$\mathcal{A} = \mathcal{A}_1 \otimes \dots \otimes \mathcal{A}_n = (L, l_0, C, Var, \Sigma, E, Inv)$$

by:

- $L = L_1 \times \dots \times L_n$ (Cartesian product of locations);
- $l_0 = (l_0^1, \dots, l_0^n)$;
- $C = C_1 \uplus \dots \uplus C_n$ (disjoint union: each component keeps its own clocks);
- $Var = Var_1 \cup \dots \cup Var_n$ (union with identification: a variable shared by several components is represented only once — this is *gluing* in the proper sense);
- $\Sigma = \Sigma_1 \uplus \dots \uplus \Sigma_n$;
- $Inv(l_1, \dots, l_n) = \bigwedge_{k=1}^n Inv_k(l_k)$;
- E follows an *interleaving semantics*: from $(l_1, \dots, l_n)$, component k may cross a transition $(l_k, g_k, a_k, r_k, u_k, l'_k) \in E_k$ as soon as its guard g_k (over $C_k \cup Var$) is satisfied by the current valuation — including the part of the valuation of Var written by the other components. The product then reaches $(l_1, \dots, l'_k, \dots, l_n)$, resets the clocks in r_k , and applies the update u_k to the *global* valuation of Var , immediately making this change visible to the other components sharing the modified variable(s).

3.3 Preliminaries (within the team - CompareVerif)

In this section we describe the tool already available when I arrived at the institute.

3.3.1 Tooling to Survey Attacks and Adversary Capabilities in ProVerif

To make exploring attacker hierarchies practical, the research team developed **CompareVerif**, a Python-based harness for ProVerif available on GitHub [4]. The core insight driving CompareVerif is that security engineers need more than a simple true-or-false output from a prover. To design resilient

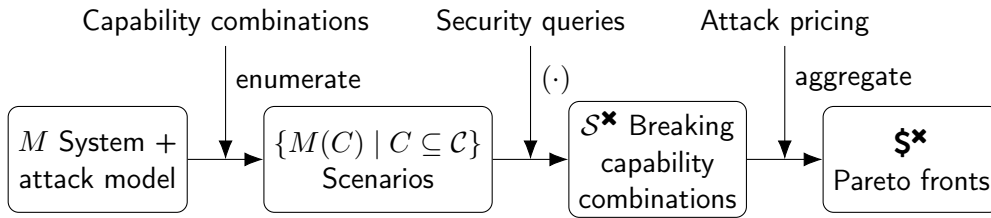


Figure 3.4: Sketch of how to derive attacker's Pareto fronts per query for priced attacker capabilities.

systems, they must understand exactly *what needs to go wrong* for an attack to succeed and to *which extent an attack is difficult*.

The mechanism involves embedding *magic comments* into standard ProVerif .pv files. These comments define priced attacker capabilities without breaking the underlying model. CompareVerif automates the workflow described in Figure 3.4, solving several common pain points:

- **Exhaustive Processing:** It automatically generates and evaluates all relevant capability combinations, eliminating brittle manual commenting/uncommenting.
- **IDE Compatibility:** Relying on comments preserves syntax highlighting and tooling support (e.g., the VS Code ProVerif plugin [18]).
- **Data Extraction:** It parses ProVerif's verbose console output, cleanly mapping results directly to the attack cost model.
- **Attack Explainability:** It generates visual attack trees linking vulnerabilities back to specific capabilities, making derivations readable for engineers.

Attack Scenario Batch Processing

Users specify their protocol and equational theory normally, injecting capabilities via the following comment syntax:

```

(** NameOfCapability [CostOfCapability CostDimensionName]
    ...
**)
```

For example, a typical data leak capability (costing 1 hack) of a variable `sensitive_var` would be injected into the main process to expose internal states to the attacker:

```

process
...
(** Data Leak of sensitive_var [1 hack]
    / out (public_channel, sensitive_var)
**)
```

Because ProVerif queries are inherently anonymous, CompareVerif extracts standard comments immediately preceding a query and treats them as the query's name:

```

(* no pw leakage *)
query attacker(pw).
```

Execution is simple. Running the preprocessor script:

```
$ python scenario_preprocessor.py [location of .pv file]
```

Yields a clean, cost-evaluated output:

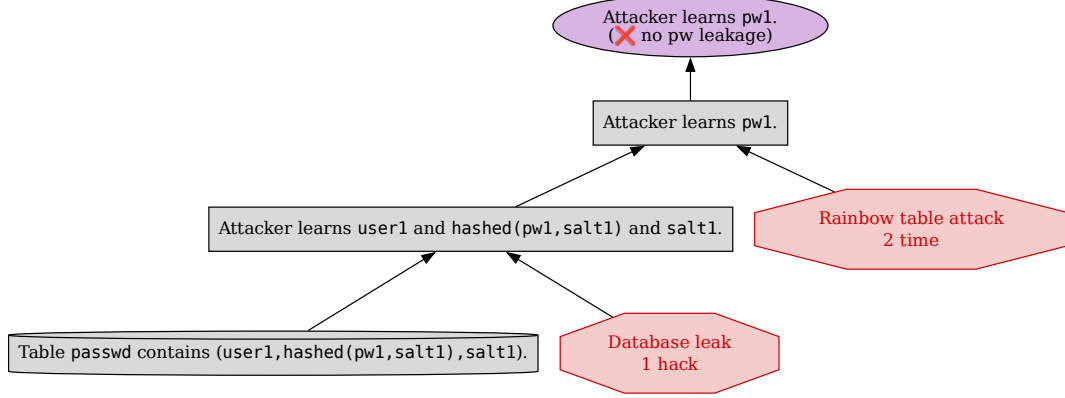


Figure 3.5: Attack tree against the password security query for the {db-leak, rainbow}-scenario on a running example `hashed_passwords_paper.pv` available on the git project

```

$ python scenario_preprocessor.py examples/hashed_passwords_paper.pv
[...]
Query: no pw leakage
  {Database leak, Rainbow table attack} (cost: 1 hack, 2 time)
  {Server compromised} (cost: 4 hack)
Query: no hashed pw leakage
  {Database leak} (cost: 1 hack)

```

Behind the scenes, the script executes four stages:

1. **Parse:** Extracts the baseline model M , capabilities $\mathcal{C}$, and prices p from the `.pv` file.
2. **Generate:** Creates distinct `.pv` scenario files for relevant capability combinations, saving them in a `_scenarios` directory.
3. **Execute:** Runs ProVerif on each scenario, logging query results and prices to a `manifest.json` file.
4. **Report:** Computes and outputs the minimal prices required to break each query.

To avoid generating $2^{|\mathcal{C}|}$ scenarios unnecessarily, the script offers flags to cull scenarios dominated by already-discovered attacks.

Extracting Attack Trees

When a security prover discovers a violation, engineers must be able to understand the attack path to validate the threat model and patch the system. To facilitate this, CompareVerif automatically generates visual *attack trees* [22].

CompareVerif provides a script that converts ProVerif’s raw logical derivations into comprehensible graphs. For instance, analyzing the {db-leak, rainbow} scenario, for a given `.pv` file available on the git project, yields the tree shown in Figure 3.5. The root node defines the violated property, red leaf nodes represent the leveraged capabilities, and gray intermediate nodes map the attacker’s logical milestones. This should be understood as a Conjunctive Attack Tree.

Under the hood, ProVerif models potential attacks using Horn clause derivation systems [6]. CompareVerif extracts these derivations and cross-references them against singleton capability scenarios. This allows the tool to accurately tag specific derivation rules with the human-readable capabilities that enabled them. Furthermore, because these trees are generated from cost-minimal scenarios, engineers are guaranteed to be looking at the cheapest valid attack path, free from unnecessary theoretical noise.

However, this tree representation abstracts away crucial details, making it difficult to soundly map the attack back to the original protocol. While ProVerif’s native Message Sequence Charts (MSCs) solve this by clearly linking attacks to protocol steps, they enforce a strict execution order and fail to represent concurrent operations. An ideal visualization would bridge this gap, combining the causal flexibility of trees with the protocol-grounding of MSCs. Furthermore, the current tree generation is difficult to translate into Timed Automata because of excessive abstraction, and provides no additional value for standard `.pv` files that lack CompareVerif’s custom annotations. We address those issues in [chapter 6](#) and [chapter 7](#).

3.3.2 Checking Protocol Security

To formally assess the protocol’s security, we query its state against a standard Dolev–Yao attacker. For instance, proving the confidentiality of the password is done via the following ProVerif query:

```
(* no pw leakage *)
query attacker(pw).
```

The command `query attacker(pw)` asks ProVerif to find any execution trace where the attacker learns `pw`. Generally, a query yields one of four outcomes:

- ✓ **Secure:** ProVerif proves the attacker cannot satisfy the query.
- ✗ **Vulnerable:** ProVerif finds an attack and generates an execution trace.
- ? **Inconclusive:** The over-approximation suggests an attack, but it cannot be reconstructed.
- ⊥ **Timeout:** ProVerif fails to terminate within the allotted time.

To generalize across similar tools, we define a generic model-checking function:

$$\text{check}: \Pi\mathcal{E} \times \mathcal{Q} \rightarrow \{\checkmark, \times, ?, \perp\}$$

where $\Pi\mathcal{E}$ represents the applied π -calculus system with its equational theory, and $\mathcal{Q}$ is the query language.

Calling an example model πe , the proven secrecy of `pw` is formalized as:

$$\text{check}(\pi e, \text{attacker}(\text{pw})) = \checkmark.$$

Chapter 4

Contributions to CompareVerif

4.1 Historical Context

When I arrived at the Institute, Benjamin Bisping and the team already had a first prototype of the tool CompareVerif. I helped with exhaustive debugging and helped to write the first documentation and formal definition of the math behind algorithm and implementation. This documentation would be further improved by everyone in the team to lead to the submission of a Conference Paper at SEFM 2026, currently under review, with CompareVerif as artefact submission. In that regard, CompareVerif is a joint product of the team.

4.2 Communication at AFADL GDR Scilog 2026 and at ACMES Seminar at TSP

Natalia Kushik and I gave a presentation at the Conference AFADL GDR Scilog 2026. The same presentation was given to a seminar at Telecom SudParis (ACMES). We explained the project with Orange, the goals, and the two big strategies we analyzed for the project, which are summarized in Figure 4.1.

CompareVerif	ProVerif and UPPAAL Pipeline
• Introduction of new model of an attacker • Augmentation with <i>prices</i> for <i>attacker capabilities</i> • Generation of Attack Trees for exhaustive analysis of attack	• Extraction of Attack Trees and augmentation with attack mitigation • Assigning the timeouts or delays for atomic mitigation • Extraction of a Timed Automaton to be further verified by UPPAAL

Figure 4.1: The two strategies for the project. Extract of the communication at AFADL GDR Scilog 2026.

During the two presentations, I gave a demonstration for the tool CompareVerif and explained the strategies we were considering to further pipeline ProVerif from UPPAAL into a timed automaton. I had the opportunity to take into account the feedback from the audiences during the questions section after the presentation, together with further discussions. This feedback was taken into account for the improvement of CompareVerif before its submission to SEFM, but also for the pipelining framework from ProVerif to UPPAAL. For example, one of the researchers at the talk recommended we make extensive use of parallel composition when building the timed automaton, which is something I followed in [chapter 7](#).

4.3 Contributing to the Conference Paper for SEFM 2026

I contributed to formulating the mathematical framework and helped write down the algorithm. I did exhaustive debugging. I wrote the first documentation and formal definition of the algorithm in [subsection 4.4.1](#). This documentation would be further improved to lead to the submission of a Conference Paper at SEFM 2026, currently under review, with CompareVerif as artefact submission.

I wrote and documented some of the first use cases of the artefact as showcased in [4.4.2](#).

4.4 Reasoning with Capabilities

4.4.1 Mathematical Framework for Capabilities

This section details the theoretical foundation I helped establish for CompareVerif, providing a rigorous mathematical formalization of attacker capability hierarchies to systematically evaluate protocol security. As outlined in [Figure 3.4](#), the core algorithm evaluates a system model M across various combinations of these capabilities. By identifying the minimal capability sets required to break specific security properties, we can effectively determine the cost of minimal attacks.

Modeling Attacker Capabilities

Let $\mathcal{C} = \{c_1, c_2, \dots\}$ be a set of *attack capability names*. We evaluate a system model parameterized by subsets of capabilities $C \subseteq \mathcal{C}$, represented as a function $M: 2^{\mathcal{C}} \rightarrow \mathbb{P}\mathcal{E}$.

Each resulting $M(C)$ is a *scenario*. The baseline scenario $M(\emptyset)$ corresponds to the standard Dolev–Yao model without extra capabilities. For any query $q \in \mathcal{Q}$, we define the sets of capability combinations where the protocol succeeds ($\mathcal{S}^\checkmark$) or fails ($\mathcal{S}^\times$):

$$\begin{aligned}\mathcal{S}_{M,q}^\checkmark &:= \{C \subseteq \mathcal{C} \mid \text{check}(M(C), q) = \checkmark\} \\ \mathcal{S}_{M,q}^\times &:= \{C \subseteq \mathcal{C} \mid \text{check}(M(C), q) = \times\}\end{aligned}$$

We assume **monotonicity**: adding capabilities to the attacker only strictly increases their power. An attacker will never fail an attack because they gained a new capability. Formally:

$$C \subseteq C' \text{ and } \text{check}(M(C), q) = \times \text{ implies } \text{check}(M(C'), q) = \times$$

This monotonicity is vital. It allows us to filter the powerset lattice down to only the *minimal* capability combinations required to break a property:

$$\text{Min } \mathcal{S}_{M,q}^\times := \{C \in \mathcal{S}_{M,q}^\times \mid \nexists C' \in \mathcal{S}_{M,q}^\times. C' \subset C\}$$

Hasse Diagram of capabilities Consider a situation where we have three capabilities $\mathcal{C} = \{c_1, c_2, c_3\}$.

Suppose there are two minimal capability combinations to break password secrecy:

$$\text{Min } \mathcal{S}_{M, \text{attacker}(\text{pw})}^\times = \{\{c_1\}, \{c_2, c_3\}\}$$

[Figure 4.2](#) visualizes this partially ordered hierarchy. It is possible to generalize CompareVerif model of the attacker with any lattice.

Potentially, this lattice diagram could be augmented into a Timed Automaton for further analysis of the timed aspects of the attack.

Pricing Attacker Capabilities

Let $\text{price}: 2^{\mathcal{C}} \rightarrow \mathcal{P}$ be a pricing function mapping capabilities to an m -dimensional cost space (typically $\mathbb{N}^m$ or $\mathbb{R}^m$). A capability combination C is *cheaper* than C' ($P \leq P'$) if its cost is lower or equal across all m dimensions. It is *strictly cheaper* ($P < P'$) if it is cheaper and strictly lower in at least one dimension.

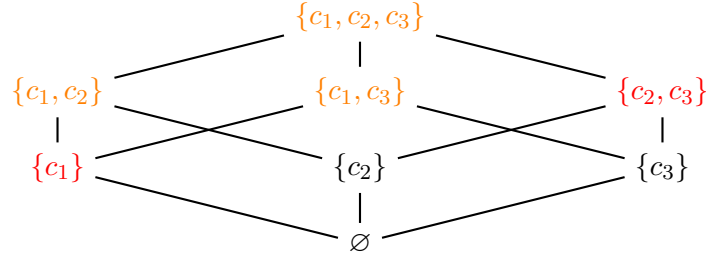


Figure 4.2: Hasse diagram of the partially ordered set of capability combinations $(2^{\mathcal{C}}, \subseteq)$ for the example. The subset $\mathcal{S}_{M, \text{attacker}(\text{pw})}^*$ where query `attacker(pw)` breaks is highlighted, singling out its minimal elements $\text{Min } \mathcal{S}_{M, \text{attacker}(\text{pw})}^*$ in red.

Assuming the pricing function is monotonic (adding capabilities never decreases cost), we can compute the **optimal cost threshold of these attacks**:

$$\begin{aligned} \text{Min } \mathcal{S}_{M,q}^* &= \{P \mid P = \text{price}(C), C \in \text{Min } \mathcal{S}_{M,q}^* \text{ and} \\ &\quad \nexists C' \in \text{Min } \mathcal{S}_{M,q}^*. \text{price}(C') < P\} \\ \mathcal{S}_{M,q}^* &= \uparrow(\text{Min } \mathcal{S}_{M,q}^*) = \{P \in \mathcal{P} \mid \exists P' \in \text{Min } \mathcal{S}_{M,q}^*. P' \leq P\} \end{aligned}$$

This provides a rigorous metric for system comparison: **Protocol A is more secure than Protocol B** if its minimal required attack budget is strictly larger. Formally, $\mathcal{S}_{M_A, q_A}^* \subset \mathcal{S}_{M_B, q_B}^*$ implies that any successful attack on A can be matched by an equally cheap or cheaper attack on B, with at least one attack on B being strictly cheaper.

Additional Hypothesis

To ensure practical usability, CompareVerif’s approach to pricing capabilities relies on an additional additivity assumption. Let $m \in \mathbb{N}$ denote the number of dimensions used to measure the cost of these capabilities. The pricing is evaluated within a discrete vector space $\mathcal{P} := \mathbb{N}^m$, where each component represents a specific cost dimension—such as computational time, required technical skills, physical access requirements, or specific actions like `hack` and `shoulder surfing` (as illustrated in 3.3.1). We assume the existence of a pricing function $p: \mathcal{C} \rightarrow \mathbb{N}^m$ that satisfies the following equation for any capability combination $C \in 2^{\mathcal{C}}$:

$$\text{price}(C) := \sum_{c \in C} p(c)$$

4.4.2 Analyzing a simple ratchet protocol

In order to showcase the efficiency of the brute-force-search strategy of CompareVerif, we showcase a simple example of an application on an elementary ratchet protocol. This protocol is a more primitive version of the double-ratchet protocol of Signal. In that regard, CompareVerif could potentially be used to analyze multiple protocols in Signal. It is a toy example. It comes with the optimal cost threshold of attacks along with multiple generated attack trees available in the following link :

https://github.com/benkeks/compareverif/blob/main/notebooks/pareto_fronts_and_generation_of_AG_simple_ratchet.ipynb

4.5 Learning from the experience with CompareVerif

With CompareVerif, we explained how to quantify the budgets that an attacker needs to break security properties, using symbolic verification. The key contribution is the structured augmentation of protocol models with models of priced attacker capabilities applied to the ProVerif ecosystem. This leads to an exhaustive analysis of attacks that the adversary may perform.

CompareVerif showcases that bruteforcing in formal methods may not be doomed to fail. Because we use `check` a total of $2^{\#C}$ times, we can track the evolution of the bruteforce search in real-time, whereas in other circumstances, the tool running the bruteforce provides the user with no feedback about the time remaining before execution finishes. The user can also control the amount of bruteforcing by modifying the model to change the value of $\#C$.

Regarding possible directions for improvement, CompareVerif is currently limited to pricing the presence of attack capabilities in a scenario. For a more refined analysis of how often an attacker uses their abilities, a richer preprocessing or a resource-aware derivation of attacks would be necessary, going beyond ProVerif. Similar limitations exist with respect to more specific timing constraints of attacks and defenses, which play a role in the context of moving target defense strategies [15]. To address both resource usage and timing constraints, it would be beneficial to have security solvers that treat attack prices and timed mitigations as built-in features. The tools in [chapter 5](#), [chapter 6](#), [chapter 7](#), lay my theoretical contributions to address those issues.

Chapter 5

Experimenting with a randomization feature on ProVerif's algorithm

In this chapter we specify a small "randomness" feature that can be added to ProVerif's list of functionalities. We define this "randomness" feature, explain its use cases, and specify how to modify ProVerif's source code to use it functionally. We call **Randomized ProVerif** the resulting modified version of ProVerif. We implement the modifications of ProVerif's algorithm to the source code, which is available on the following github :

https://github.com/NicolasMatei/randomized_proverif

5.1 Modifying ProVerif's Attack Reconstruction Algorithm

Since the construction of the Horn Derivation Proof Tree depends on the order of the Horn clauses generated at the beginning, we modified ProVerif to randomize the order of their appearance once they have been generated from the system specification.

The consequence of this simple modification is that the Horn Derivation Resolution does not follow the same resolution path every time ProVerif is executed. Consequently, the Attack Reconstruction Algorithm described in [subsection 3.2.2](#) will not work with the same resolution trees at each execution of ProVerif. Running **Randomized ProVerif** multiple times can yield different attacks. This randomized approach allows for a much more exhaustive attack generation process.

To avoid overwhelming this document, we leave the enumeration of the specific source code modifications made to ProVerif 2.05 to the **Randomized ProVerif** GitHub repository. Furthermore, in [chapter 6](#), we detail a Python script, `generate_cag.py`, which makes full use of this modification by automating the repeated execution of **Randomized ProVerif** to systematically and exhaustively generate Attack Trees.

5.2 Simple Example

In order to showcase the usefulness of the new feature we showcase an application on the toy example from [subsection 3.2.3](#) in [Figure 5.1](#).

<pre>Derivation: goal attacker(success[]) clause 19 attacker(success[]) clause 22 attacker(hash(nonce[],seed[])) clause 20 attacker(secretcondition[]) clause 23 attacker(nonce[])</pre>	<pre>Derivation: goal attacker(success[]) clause 18 attacker(success[]) clause 21 attacker(enc(nonce[],sk[])) clause 20 attacker(secretcondition[]) clause 23 attacker(nonce[])</pre>
---	--

Figure 5.1: Reduced output of **Randomized ProVerif** executed on the toy example from `how_many_attack.pv` from [figure 3.2.3](#). Two different attacks against the same queried property are found by executing **Randomized ProVerif** twice. Use `-set explainDerivation false` option to print the attacks as Horn Derivation Proof Trees.

5.3 Possible Improvements to the modified algorithm of Proverif

This small experiment showcased that modifying ProVerif's source code is not as scary as it seems. Here are two ways this can be improved.

We only apply a random permutation once. Potentially we could apply permutations multiple times in the Horn Derivation Algorithm. Potentially **Randomized ProVerif** may find new attacks this way.

Additionally, we apply a random permutation on the list of formulas. However, we could also apply other transformations such as removal of a given number of Horn Clauses from the list.

Chapter 6

Improving the Tree Generation of ProVerif

This chapter presents an automated toolset designed to improve attack tree generation in ProVerif by transforming Horn derivation proof trees into fine-grained Conjunctive Attack Graphs (CAG). To support our proposed pipeline, ensure full reproducibility, and provide execution scripts along with example models, the complete implementation is available in the following open-source repository:

https://github.com/NicolasMatei/Attack_Tree_Generation

6.1 Context and Motivation

When ProVerif identifies a security property violation, its Attack Reconstruction Algorithm generates an attack trace. Typically, this trace is presented either as sequential terminal output or as Message Sequence Charts (MSCs), for instance by running:

```
$ proverif -html resultats_attaques mon_protocole.pv
```

Although MSCs are derived from the underlying Horn Derivation Proof Tree, they linearize the execution and lose the natural commutativity between independent attack steps. Conjunctive Attack Trees offer a richer alternative by preserving these sequential nuances.

Currently, tools like CompareVerif can parse these Horn derivation trees into `json` format and render them as graphs using `graphviz`. However, these trees focus exclusively on the attacker’s internal computations (i.e., term construction via Horn clauses). They completely abstract away the protocol’s agents and the underlying Applied-Pi Calculus semantics.

To pipeline these attacks into UPPAAL as timed automata, it is strictly necessary to link the attacker’s actions back to the protocol’s execution states. Bridging this gap is our main motivation. We want to develop a general-purpose tool capable of generating protocol-aware Conjunctive Attack Trees directly from any `.pv` file.

6.2 Hypotheses

To ensure that the abstractions presented here are sound, we impose the two assumptions stated in [subsection 3.2.5](#). Because we rely on pi-ids, if two processes in parallel use the same pi-id, we cannot guarantee how the resulting tool will respond.

6.3 Definitions

Formal Definition: Protocol Elements: Indices, Direction, and Nodes

Input/Output Indices. Let $\Pi \subset \mathbb{N}$ be a set of *communication indices* (π -ids). Each index $p \in \Pi$ will identify a specific input (**input**) or output (**output**) point of the protocol, denoted $\{p\}$ in the descriptions.

Direction. We denote $\text{IO} = \{\text{input}, \text{output}\}$ as the set of the two possible communication directions:

- **input** (yellow node): message received *from* the attacker;
- **output** (blue node): message sent *to* the attacker.

Node. A *node* ν is either:

1. a **goal node** (*goal*, red), labeled by a fact g ; such a node is denoted $\nu = \text{Goal}(g)$;
2. an **input/output node** (*io-node*, yellow or blue), labeled by a triplet $(io, p, t) \in \text{IO} \times \Pi \times \mathcal{T}$; such a node is denoted $\nu = \text{IO}(io, p, t)$.

Each node also carries a (possibly empty) list of children $\text{children}(\nu) = [\nu_1, \dots, \nu_k]$, where each ν_i is itself a node (**obligatorily** an input/output node; only the root can be a goal node).

Formal Definition: Conjunctive Tree

A *conjunctive derivation tree* T is a pair $T = (\nu_{\text{root}}, \text{children})$ where:

- $\nu_{\text{root}} = \text{Goal}(g)$ for a fact g which we define as the queried property (the root is always a goal node);
- children associates to each node of the tree the (finite) list of its children, as described above;
- the structure induced by children is acyclic and finite (it is indeed a tree: each node other than the root has a unique parent).

We denote $\text{io}(\nu)$, $\text{pi}(\nu)$ and $\text{term}(\nu)$ as the respective projections io , p and t of a node $\nu = \text{IO}(io, p, t)$.

The term "conjunctive" is used because the conditions act as a logical AND. Informally, the attacker semantics require all child nodes to be satisfied before their parent node can be satisfied. Finally, we expand this to define more complex attack graphs with multiple roots.

Formal Definition: Conjunctive and Disjunctive Attack Graph/Forest

We enrich Definition 4 (Node) with a third type: a **gate node** (*gate*), denoted $\text{Gate}(r)$ with $r \in \{\text{AND}, \text{OR}\}$ (green if $r = \text{AND}$, orange diamond if $r = \text{OR}$). A *conjunctive and disjunctive attack graph* is a tree in the sense of Definition 5 (Conjunctive Tree) admitting multiple roots, in which any Goal or IO node having at least two children connects them via a unique intermediate $\text{Gate}(r)$ node. Furthermore, a gate node *always* verifies $|\text{children}(\text{Gate}(r))| \geq 2$. Such a tree is called *conjunctive* if all its gate nodes are AND gates; it is called *conjunctive and disjunctive* if it contains at least one OR gate.

6.4 Specification and Implementation of the generation of attack trees

6.4.1 Specification

1st Step (Extraction of the Horn Derivation Proof Tree) : Similarly to CompareVerif's tree extraction, we start by extracting the Horn Derivation Proof Tree, from which the Attack Reconstruction Algorithm has found an attack, as raw text. This can be extracted by executing the following command:

```
$ proverif -set explainDerivation false [file.pv]
```

Moreover to have a bigger amount of trees and exhaustive tree generation, we can use **Randomized ProVerif** multiple times (perhaps 10 times), in order to get more attack trees. By executing 10 times, we can expect in most cases to have all the trees we can get using Randomized ProVerif. However using Randomized ProVerif is not necessary and we can extract Trees by executing original ProVerif only once.

The Algorithm will then apply multiple transformations to the tree. Each of these transformations will hold the informal property of faithfulness, meaning each transformation will never remove information from the initial tree, but can only add more information. This is mostly an informal justification of soundness.

2nd Step (Adding Descriptions for each Horn Clause used). We extract descriptions of the Horn clauses in the tree by executing ProVerif with the `-set verboseClauses explained` option activated. An extract of such descriptions extracted from ProVerif's output is given in [Figure 6.1](#).

Clause 21: `attacker(secretcondition[])`

(The message `secretcondition[]` may be sent to the attacker at output {8}.)

Clause 22: `attacker(secretcondition[]) && attacker(x_4) -> attacker(enc(x_4,sk[]))`

(If the message `x_4` is received from the attacker at input {9},
the message `secretcondition[]` is received from the attacker at input {10},
then the message `enc(x_4,sk[])` may be sent to the attacker at output {12}.)

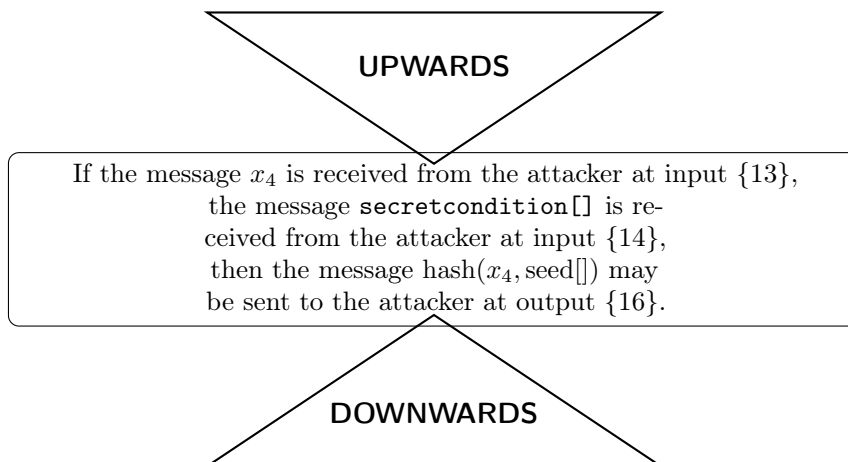
Figure 6.1: Extract of the output of ProVerif on the toy example from [subsection 3.2.3](#) after applying the `-set verboseClauses explained` option. We start the algorithm by replacing the clauses in the Horn Derivation Proof Tree with these descriptions.

For example, if ProVerif interprets the following description for a Horn clause :

Clause 23: `attacker(secretcondition[]) && attacker(x_4) -> attacker(hash(x_4,seed[]))`

(If the message `x_4` is received from the attacker at input {13},
the message `secretcondition[]` is received from the attacker at input {14},
then the message `hash(x_4,seed[])` may be sent to the attacker at output {16}.)

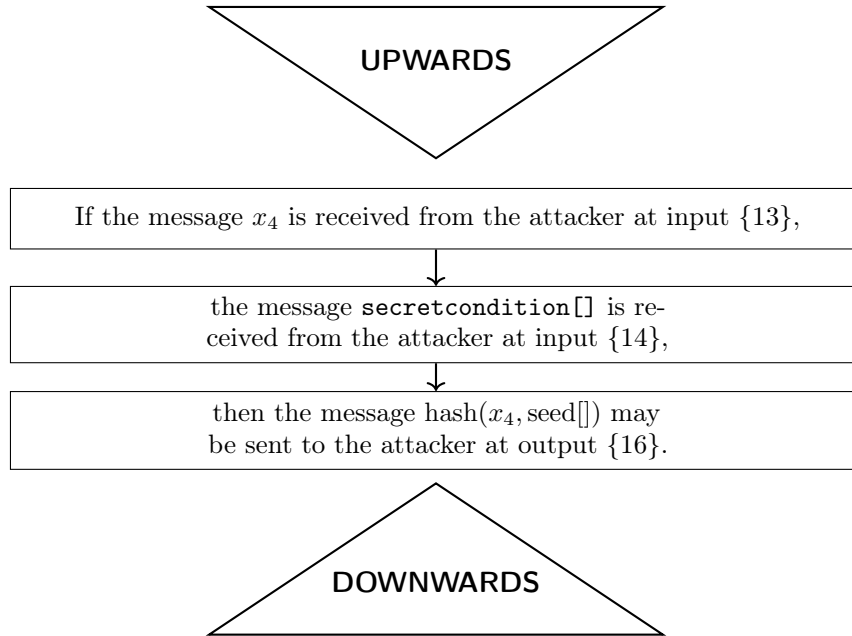
Then, if clause 23 appears in the horn derivation Proof Tree as a node, we should have the following node interacting with the tree in the following way:



Where upwards and backwards both represent the rest of the already existing tree.

3rd Step : (Decomposition of nodes in intermediary nodes) This is the most important part of the algorithm. Now that we have extracted a concrete description of each node from ProVerif's output which make reference to the numerical parsing of the system in ProVerif, we can decompose the nodes in intermediary steps.

We transform it in the following way, where each node is split:



4th Step : (Removing duplicates) : Occasionally a Horn Derivation Proof Tree might use the same clause for multiple leaves. ProVerif parses it as a "duplicate" tree. For each duplicate tree, we go and fetch the correct description. This is not algorithm-wise related and more an implementation problem.

5th Step (Printing the .pdf file) : We use graphviz to transform the resulting json file into a pdf file. In order to have a user-friendly output, we impose that the output trees follow the definition of Conjunctive Attack Trees from [section 6.3](#). Yellow for input, blue for output, and red for goal.

6.4.2 Implementation

The tree generation along with an example is implemented in the git project.

The entire pipeline is fully automated and can be executed via a single script (see [Figure 6.2](#) for illustration). From the project folder, you can launch the process using `generate_cag.py`:

```
$ python3 generate_cag.py <file>.pv -classical
```

or, alternatively if you want to try generating more trees,

```
$ python3 generate_cag.py <file>.pv -randomized 10
```

The resulting JSON and PDF files are automatically generated in the `<file>._resolus/` and `<file>._pdf/` directories, respectively.

6.4.3 Results

When executing the new attack tree generation on the toy example from [3.1](#), we obtain three conjunctive attack trees showcased in [Figure 6.3](#). These three trees will be further transformed into a forest. Observe the resulting tree follows the definition from [section 6.3](#).

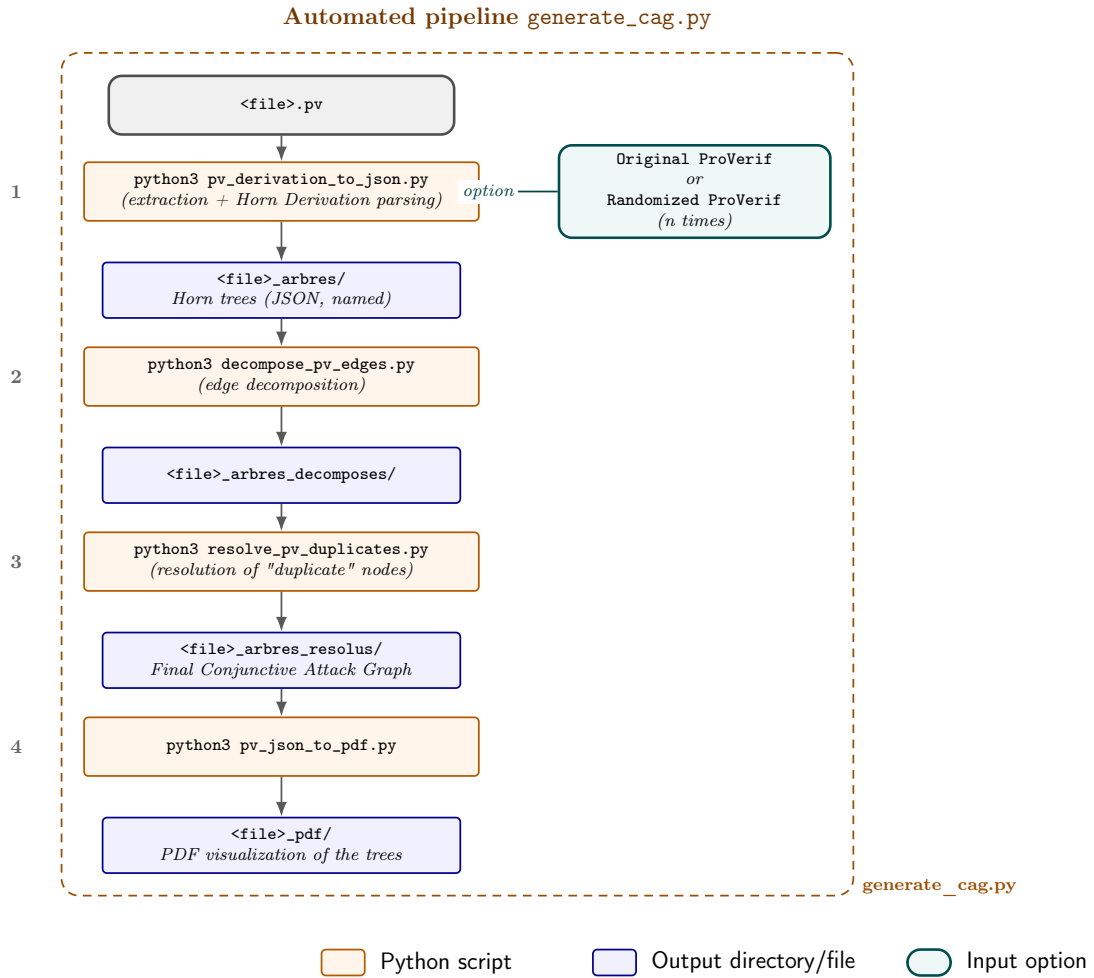


Figure 6.2: Automated pipeline of `generate_cag.py`: after extracting the Horn Derivation Proof Trees (`pv_derivation_to_json.py`), long edges are decomposed into node sequences (`decompose_pv_edges.py`) and duplicate nodes are resolved (`resolve_pv_duplicates.py`), yielding the final Conjunctive Attack Graph, which `pv_json_to_pdf.py` then renders as a PDF.

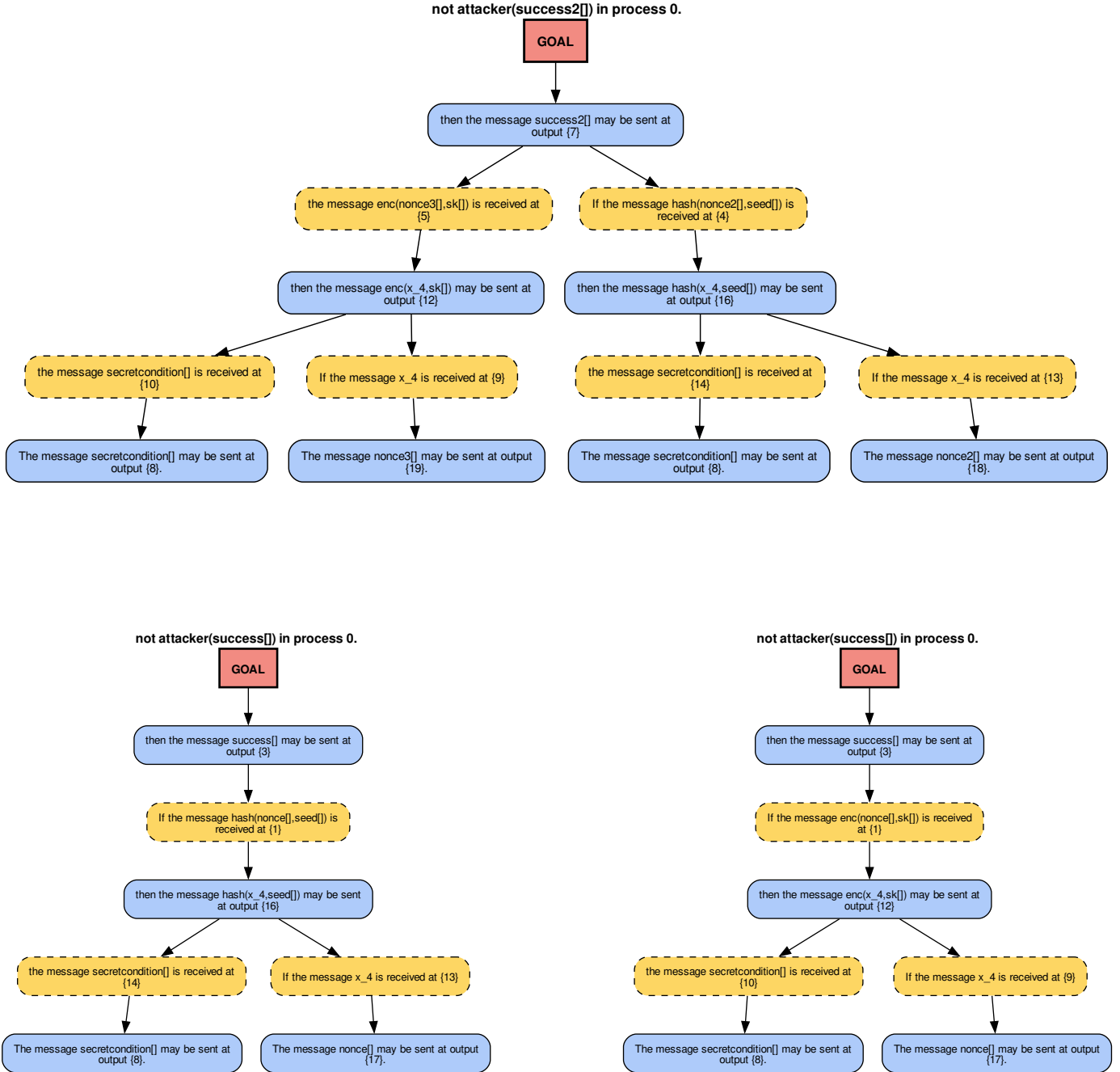


Figure 6.3: Output tree from the Attack Tree Generator extracted from the toy example from [subsection 3.2.3](#). The trees satisfy the formal definition from [section 6.3](#). Notice we have two trees for the same queried property because we used Randomized Proverif.

6.5 Further transforming the directory of Conjunctive Trees into a Master Forest

Now that we have a conjunctive attack tree generator that better abstracts away the relationship between the attack and the protocol, it is natural to ask whether this gained precision can be used to combine the trees into a forest with conjunctive and disjunctive nodes, similar to the tree found in [2].

This is what we call generation of the master forest. We can also call it an Attack Graph. In this section, we describe an algorithm for the generation of the master forest, built upon the previous algorithm. We explain the specification, the implementation, and an illustrative application example generated from the trees from Figure 6.3.

6.5.1 Specification

First Step : We transformed the resolved json trees so that they match the formal definition from section 6.3. This way, applying the transformations below will be easier.

Second Step : We group the attack trees that prove the same queried property at the root. Within a group, the derivations are merged top-down. For each node position, we collect, over every derivation that reaches that node, the set of children it uses there. If all derivations agree on this set, the node is marked conjunctive (AND): every child is necessary. If the sets differ between derivations, the node is marked disjunctive (OR) and we add the rest of the sub-tree.

Third Step: We walk the tree from the leaves to the root. At each node, we compute a signature from its identity, its AND/OR relation, and the signatures already computed for its children. If a node with the same signature already exists elsewhere in the graph – even under a different goal – we reuse it instead of drawing it again. Repeating this sharing up to the root turns the forest into a single graph.

Fourth Step : We transform it into a .pdf file.

6.5.2 Implementation

The implementation is available on the same git project. Because it builds upon the Attack Tree generation, the previous tool needs to have been run first. After `python3 generate_cag.py` has been run on `how_many_attack.pv`, the user can execute the following command :

```
$ python3 run_master_tree_pipeline.py how_many_attack_arbres_resolus arbre_maitre.pdf
```

The implementation is illustrated in Figure 6.4.

6.5.3 Results

Applying the generation of a forest to the three trees generated previously and showcased in Figure 6.3, originally built from the toy example from subsection 3.2.3, renders the forest showcased in Figure 6.5.

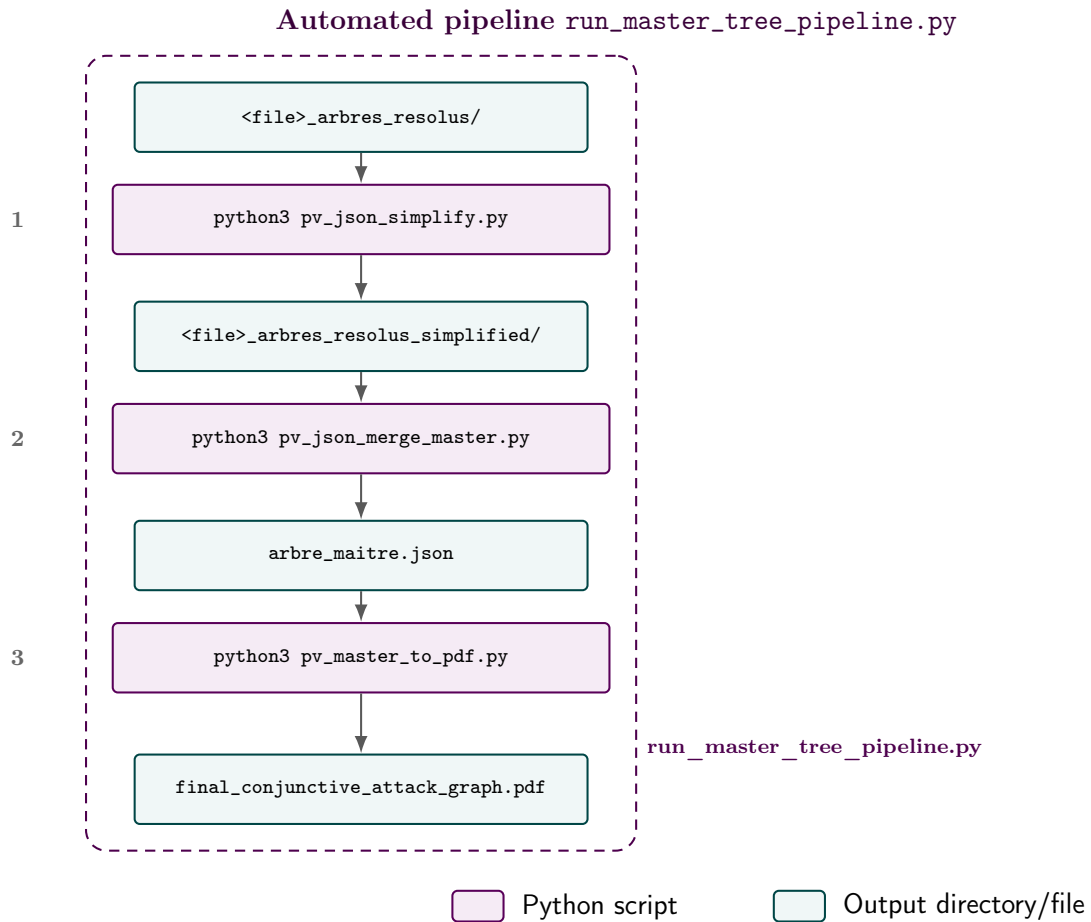


Figure 6.4: Automated pipeline of `run_master_tree_pipeline.py`: the resolved Conjunctive Attack Graphs are simplified (`pv_json_simplify.py`), merged by goal to uncover AND/OR alternatives (`pv_json_merge_master.py`), and rendered into the final Conjunctive and Disjunctive Attack Graph (`pv_master_to_pdf.py`).

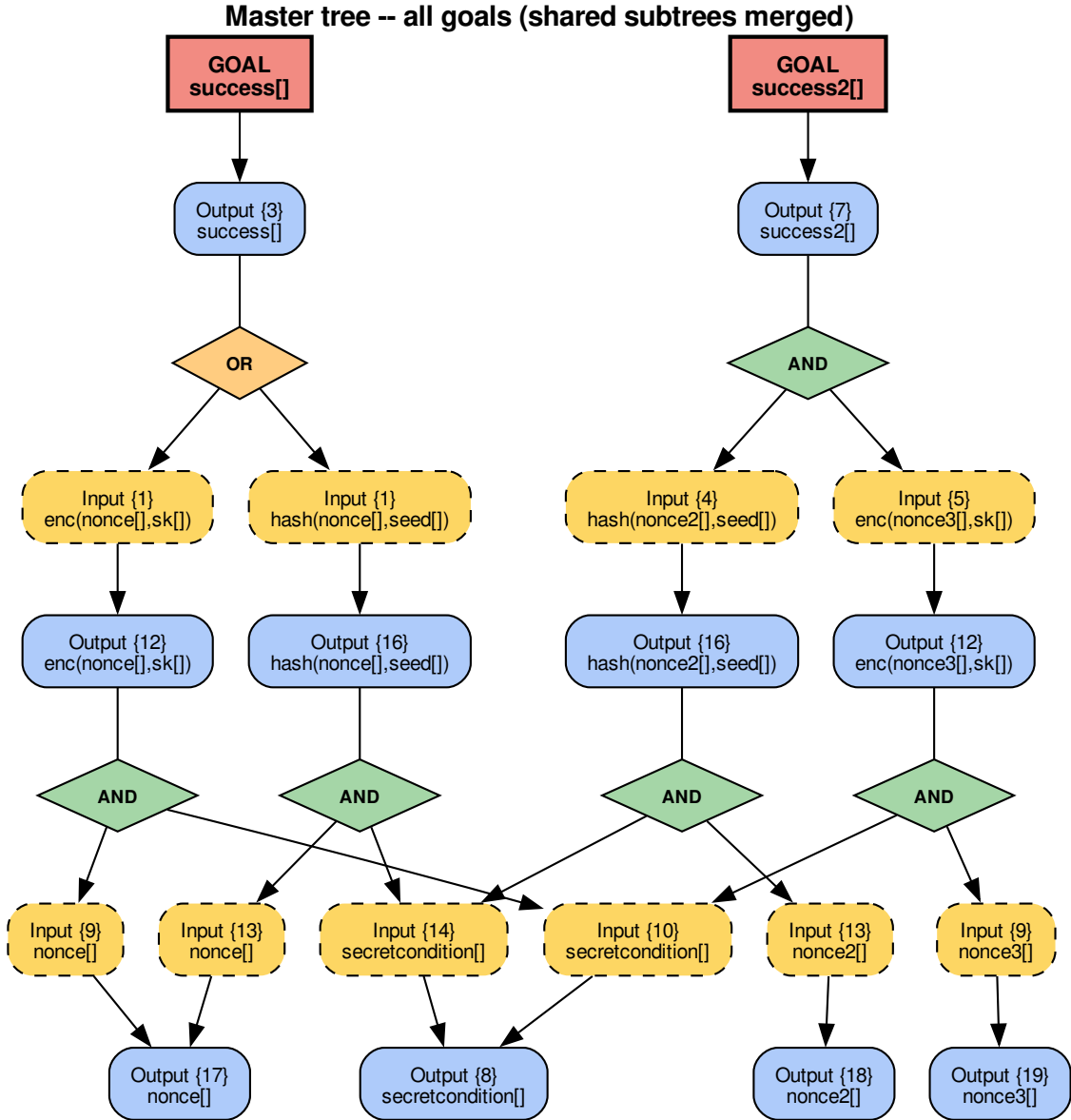


Figure 6.5: Example of a master forest generated by the presented tool on the toy example from [subsection 3.2.3](#). The colors match the formal definition of Conjunctive and Disjunctive forest from [section 6.3](#). Yellow for input, blue for output, red for goal.

6.6 Comparing the new Tree Generation with CompareVerif's Tree generation

Informal semantics of the tree: In CompareVerif's tree generation, the nodes are a reference to the computations of the attacker, while the new Tree generation make clear reference to the execution of the protocol along with identification numbers for each steps. In ProVerif's mathematical framework, there are two logics. There are terms which, along with the equational theory, represent the computation of the attacker. Then there are extended processes which represent the execution of the protocol, and of which the logical derivation rules call upon the notion of terms. CompareVerif's Tree generation is mostly related to the computation of the terms, while the new generation talks about the processes.

Topology of the graph: Because CompareVerif adds nodes for capabilities, the topology of the graph might be slightly different.

Treatment of Capabilities: The new generation of attack graph has not been tailored to parse the capabilities of CompareVerif. In a way, capabilities become atomic steps in the protocol execution, and these steps appear as nodes in the graph whenever it is an input or an output, while in CompareVerif's tree generation they would appear as leaves.

Further transformation into an Attack Graph with Conjunctions and Disjunctions: The improved abstraction of the attack trees will enable the creation of more complex structures. We can further transform the Conjunctive Attack Trees into more complex graphs with conjunctions and disjunctions because the Attack Trees abstract information better.

Necessary Hypotheses: We have not investigated yet to what extent the Attack Tree generation fails if the system does not satisfy the constraints in [subsection 3.2.5](#). CompareVerif does not assume anything upon the system.

Chapter 7

Towards Timed Proverif

We propose an abstract framework for pipelining ProVerif to UPPAAL. The main contribution is having established a general strategy to do this in a harmonious way. As output for ProVerif, we use the Horn Derivation Proof Trees after the Attack Reconstruction Algorithm has been applied to them.

Furthermore, this approach is decomposed into three independent parts. In [Figure 7.1](#), we can see a reduced representation of the framework. More specifically, this is the final stage of the framework, where we combine three timed automata in the form of .xml files that were generated independently by three **independent** parts of the framework. The advantage of having a framework decomposed into three parts is that the researcher will be able to explain the correctness of these three parts independently of one another, and the engineer can modify the software implementation code of one of the three parts without touching the other two parts. This facilitates experimentation with the proposed theoretical tools. The modularity and decomposition into three independent parts facilitates the maintenance and modification of the code.

This framework builds upon tools from the previous sections, most notably Randomize Proverif from [chapter 5](#) and the Attack Tree Generator from [chapter 6](#).

With further investment, the foundational work presented in this chapter could possibly be rendered into a fully automated and functional tool that would extend the analysis of security properties in Proverif, in the symbolic model, with timed aspects of the execution of the protocol.

Unfortunately, since the definition of the developed tool is not yet finished, we have no implementation yet. However we showcase how a timed system built upon the toy example from [subsection 3.2.3](#) would look like in the following git project :

https://github.com/NicolasMatei/uppaal_system_example

We give both the final generated .xml file along with screenshots to visualize, if the reader does not want to go through the trouble of installing UPPAAL.

Final UPPAAL Timed-Automaton Generation

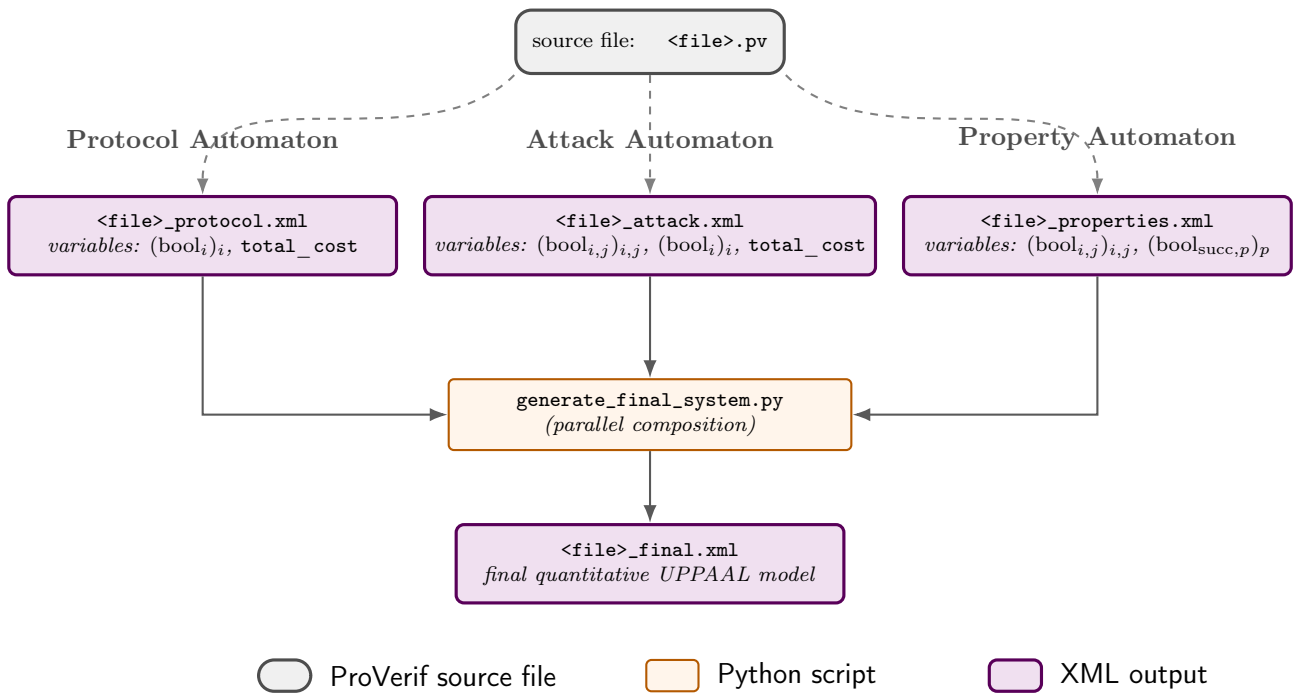


Figure 7.1: Final Phase: starting from the same source file `<file>.pv`, the earlier stages of the pipeline produce three timed automata. The script `generate_final_system.py` then combines the three `.xml` files via parallel composition to produce the final automaton.

7.1 Assumptions and Metrics

7.1.1 Modeling assumptions

To guarantee the correctness of the framework, we need to call upon the hypotheses stated in [subsection 3.2.5](#). With additional work, it should be possible to dispense with these two assumptions.

7.1.2 Extraction of atomic actions and legend

To exploit the system described in the `.pv` file, we use the parsing tool `extract_pi.py` from [Figure 3.2](#). We recall the output of the parser on the toy example of [subsection 3.2.3](#) is showcased in [Figure 3.2](#).

7.1.3 Modeling the protocol's execution tree

From the parser's output, we model the protocol's execution as a directed acyclic graph (DAG) denoted $\mathcal{G}$, often loosely referred to as an "execution tree," or "protocol tree," or "protocol execution tree."

For an example of the execution graph associated with the toy example from [subsection 3.2.3](#), see [Figure 7.2](#).

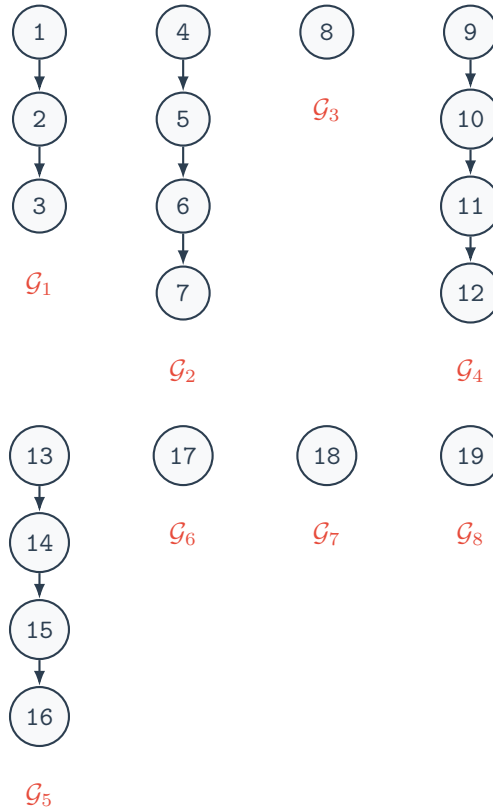


Figure 7.2: Execution graph $\mathcal{G}$ on the toy example of [subsection 3.2.3](#), partitioned into eight disjoint connected components $\mathcal{G}_1, \dots, \mathcal{G}_8$. Each arrow represents an edge of E , i.e. a sequential succession relation between two atomic actions. Note that in the current graph, conditional atomic actions such as `if`, `then`, `else` are poorly represented

Formal Definition: Execution Graph $\mathcal{G}$

Let $\mathcal{G} = (V, E)$ be the protocol's execution graph, where:

- V is the set of vertices, corresponding to atomic action identifiers (pi-id);
- $E \subseteq V \times V$ is the set of directed edges, representing the sequential control flow.

Properties following from the assumptions(subsection 7.1.1):

1. $\mathcal{G}$ contains no cycle (consequence of Assumption 1);
2. $\mathcal{G}$ is a *forest* composed of n disjoint connected components (consequence of Assumption 2):

$$V = \biguplus_{i=1}^n V_i \quad \text{and} \quad E = \biguplus_{i=1}^n E_i,$$

where each subgraph $\mathcal{G}_i = (V_i, E_i)$ represents the sequential execution of an isolated parallel process. Note that $\mathcal{G}_i$ is generally a tree, except in the presence of a non-terminal conditional convergence (see point 3 below), in which case $\mathcal{G}_i$ remains a DAG but locally loses the tree property (a convergence vertex w then has two predecessors).

Additional property:

3. **Convergence of conditional branches:** let $u \in V_i$ be a conditional action (**if cond then P else Q**), generating two disjoint paths v_{then} and v_{else} such that $(u, v_{then}), (u, v_{else}) \in E_i$. We have yet to give a correct definition of this step.

Applying this definition to the reference ProVerif trace, the set V groups together the identifiers 1 through 19. The graph is partitioned into eight connected components ($\mathcal{G}_1$ to $\mathcal{G}_8$), created by the `|` separators.

The execution graph is an essential object in the construction of the timed automaton embodying the protocol.

7.1.4 Generation of attack metrics

For the part concerning the generation of the attack automaton and the property automaton, we use an intermediate file for attack metrics, which is generated from the trees produced by the attack tree generator of Chapter 6. We explain how this file works by illustrating it with [Figure 7.3](#).

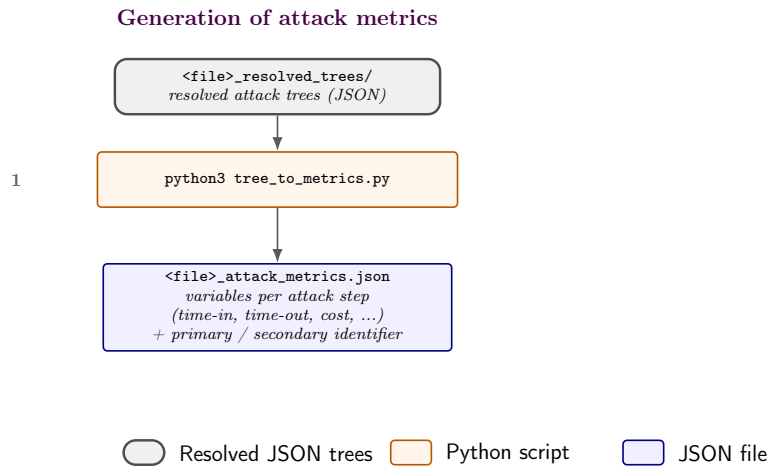


Figure 7.3: **Generation of attack metrics.** The script `tree_to_metrics.py` traverses the resolved attack trees and produces , associating with each attack step free variables to be specified, as well as a primary identifier (the line of the ProVerif parsing where the step occurs) and a secondary identifier.

Extraction and deduplication of attack steps

The script `tree_to_metrics.py` takes as input the set of resolved derivation trees produced by the tool for attack tree generation from [chapter 6](#), and extracts, for each *attack step* encountered, a set of free metrics to be instantiated.

Formal Definition: Security Queries and Multiple Derivations

Let $Q = (q_1, \dots, q_m)$ be the sequence of security queries submitted to ProVerif for which at least one attack derivation was found by the generation of attack trees from [chapter 6](#). An example of such a queried property would be `query attacker(pw)`.

We impose that the attack found for each query is different. If Randomized Proverif has been used to generate the attacks, then it is possible that some of the queries repeat themselves, that is we can have $q_i = q_j$ with $i \neq j$. If the original version of Proverif has been used then we know that $i \mapsto q_i$ is injective.

We readapt the definition of the conjunctive attack trees from [chapter 6](#) for the sake of the framework.

Formal Definition: Resolved derivation trees from [chapter 6](#)

Let $Q = \{q_1, \dots, q_m\}$ be the set of security queries submitted to ProVerif for which an attack derivation was found. For each $q_k \in Q$, we denote

$$T_k = (N_k, r_k, \rightsquigarrow_k)$$

the associated resolved derivation tree, where N_k is its set of nodes, $r_k \in N_k$ its root (the goal fact, of type `attacker(...)`), and $\rightsquigarrow_k \subseteq N_k \times N_k$ its parent-child relation. Each non-root node $n \in N_k \setminus \{r_k\}$ corresponds to a clause instance of the derivation (a **step** or a **premise**, in the sense of ProVerif's output) and carries:

- a textual description `desc(n)`, derived from the underlying ProVerif clause;
- a primary identifier $\iota(n) \in V$, extracted from `desc(n)` (the integer in braces at the end of the description, e.g. `{7}`), and coinciding with the `pi-id` of the corresponding vertex in the execution graph $\mathcal{G} = (V, E)$ defined in [subsection 7.1.3](#).

The root r_k is excluded from extraction: it represents the goal fact itself, not an atomic action of the protocol.

Formal Definition: Attack steps

Let $S = \bigsqcup_{k=1}^m (N_k \setminus \{r_k\})$ be the (multiset) union of all non-root nodes, across all queries. Two nodes $n, n' \in S$ represent the *same* attack step when `desc(n) = desc(n')`.

We write Ω for the resulting set of **distinct attack steps**.

For a node n , we write the convention $\iota(n) = \iota(\omega(n))$.

Formal Definition: Primary and secondary identifiers

For $i \in V$, let $\Omega_i = \{\omega \in \Omega \mid \iota(\omega) = i\}$ be the subset of unique steps with primary identifier i . These are the sets of attack steps that occur in the same atomic action, of `pi-id` i , in the protocol.

We define a function $\sigma : \Omega \rightarrow \mathbb{N}$, which provides secondary identifiers to attack steps. We impose that $\sigma|_{\Omega_i}$ is injective. This means that two distinct attack steps with the same primary identifier, never have the same secondary identifier.

Formal definition of the metrics file

The file `<file>_attack_metrics.json`, generated by `tree_to_metrics.py` serializes, for each unique step $\omega \in \Omega$, a set of free variables intended to be filled in by the user before the generation of

the timed automata.

Formal Definition: Attack metrics file

The attack metrics file is given by a function

$$\mu : \Omega \longrightarrow \mathbb{N} \times \mathbb{N} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}, \quad \mu(\omega) = (\iota(\omega), \sigma(\omega), w_s(\omega), w_f(\omega), c(\omega), \tau(\omega)),$$

where, for each step $\omega \in \Omega$

- $w_s(\omega)$ (**weight_success**) and $w_f(\omega)$ (**weight_failure**) are the relative success and failure weights of the attempt, used to weight the probabilistic choice (*branchpoint*) modeling the outcome of the step in the **Attack Automaton**;
- $c(\omega)$ (**cost**) is the step's contribution to the global cost variable **total_cost**, incremented when an attempt of this step reaches completion;
- $\tau(\omega)$ (**time**) is the duration that must elapse, once the attempt has begun, before its outcome (success or failure) can be determined — implemented by a clock constraint local to the step in the **Attack Automaton**.

This is implemented by `tree_to_metrics.py`.

7.1.5 Global variables and inter-component dependencies

The three automata generated by the pipeline (Figure 7.1) are not independent. Before detailing the construction of each automaton individually, we list here the complete inventory of these global variables, observed directly in a generated instance of the final system, together with their producer and their consumers.

Table 7.1 summarizes this producer/consumer structure, which is exactly the interdependency depicted by the dependency arrows added to Figure 7.1.

Variable family	Declared as	Produced by	Consumed by
$(\text{bool}_i)_{i \in V}$	<code>bool_i</code>	Protocol Automaton	Attack Automaton
<code>total_cost</code>	<code>int total_cost</code>	Protocol & Attack Automata	UPPAAL queries
$(\text{bool}_{i,j})_{(i,j) \in \Omega}$	<code>bool_atk_i_j</code>	Attack Automaton	Property Automaton
$(\text{bool}_{\text{succ},p})_{p \in \mathcal{Q}}$	<code>bool_<query>_success</code>	Property Automaton	UPPAAL queries

Table 7.1: Global variables of the composed system $\mathcal{A}_{\text{final}}^+$, and the producer/consumer relation that couples the three automata. What we mean by "produced by" is that the variables are assigned and modified in the automaton. What we mean by "consumed by" is that the variables are used for guards and conditions in the automaton, or that they would be used finally to query properties in UPPAAL with these variables.

7.2 Generation of the automaton

7.2.1 Generation of the protocol automaton

Generation of the protocol automaton is explained in Figure 7.4.

Protocol metrics file

The script `generate_blank_metrics.py` runs `extract_pi.py` on `<file>.pv`, collects every `pi-id` appearing in its output — i.e. exactly the vertex set V of the execution graph $\mathcal{G} = (V, E)$ (subsection 7.1.3), since a `pi-id` is by definition assigned to each atomic action — and builds a blank metrics record for each of them.

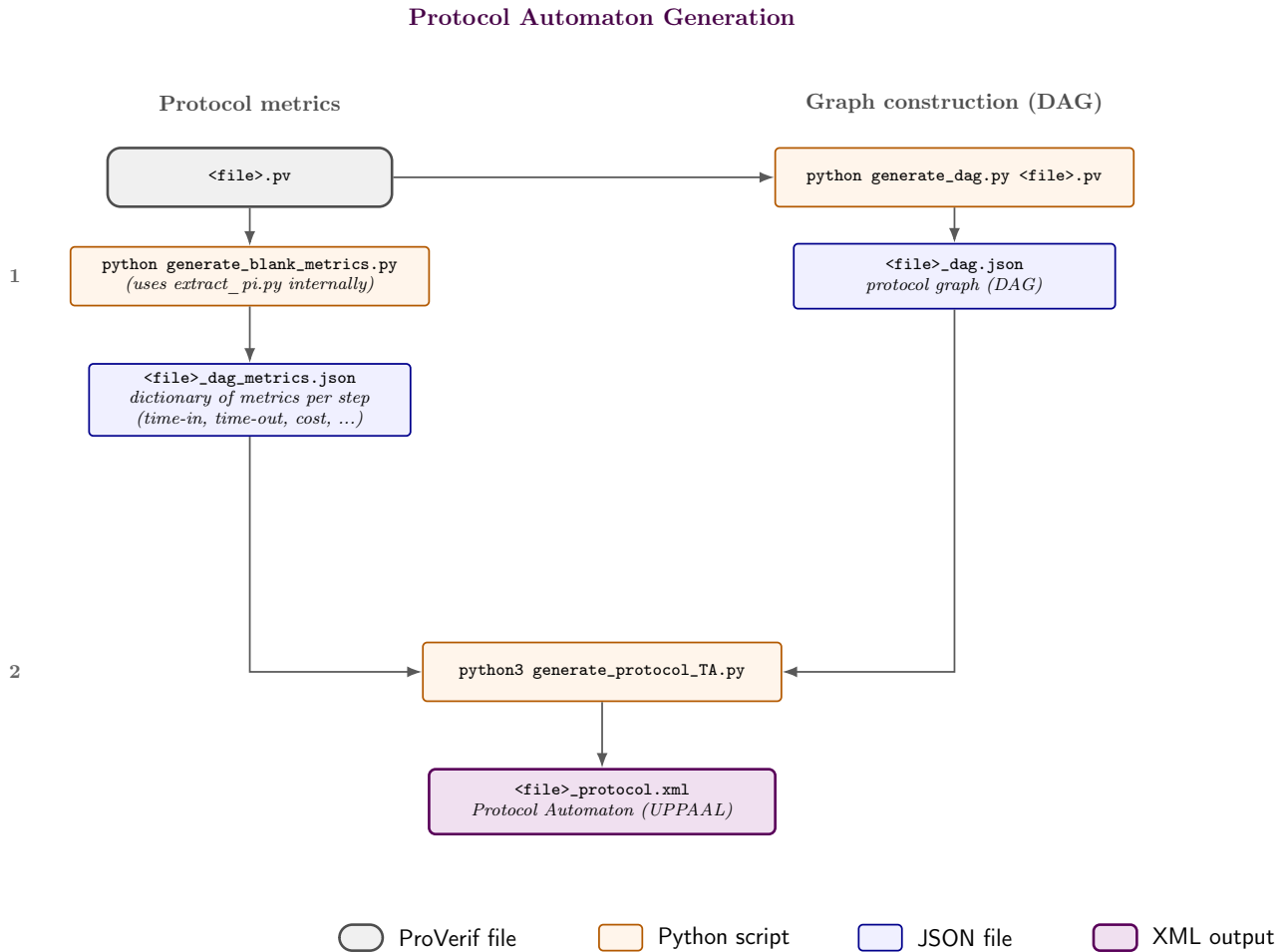


Figure 7.4: Generation of the Protocol Automaton. From a source file `<file>.pv`, the left branch extracts, via `generate_blank_metrics.py` (which internally relies on `extract_pi.py`), a dictionary `<file>_dag_metrics.json` associating with each protocol step free metrics to be specified (entry time, exit time, cost, etc.). The right branch constructs, via `generate_dag.py`, the protocol's directed acyclic graph (DAG) `<file>_dag.json`. The two JSON files are then combined by the script `generate_protocol_TA.py`, which produces the final timed automaton `<file>_protocol.xml`.

Formal Definition: Protocol metrics file

The protocol metrics file is given by a function

$$\nu : V \longrightarrow \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}, \quad \nu(i) = (\gamma(i), \delta(i)),$$

where, for each step $i \in V$:

- $\gamma(i)$ (**cost**) is step i 's contribution to the global cost variable `total_cost`, incremented when the protocol's execution token leaves i for its successor;
- $\delta(i)$ (**time**) is the duration the token must dwell at i — i.e. the value of the local clock guard/invariant governing the transition out of i in the Protocol Automaton (Figure 7.4).

This is implemented by `generate_blank_metrics.py`.

One automaton per connected component

The execution graph $\mathcal{G}$ decomposes into n disjoint connected components $\mathcal{G}_1, \dots, \mathcal{G}_n$, one per top-level sequential process. `generate_dag.py` restricts each component to a *simple path*. Branching introduced by `if/then/else` is not yet reflected. Writing $\mathcal{G}_c = (V_c, E_c)$ for the c -th component, rooted at $r_c \in V_c$, its vertices are totally ordered as a chain

$$r_c = v_1^c \rightarrow v_2^c \rightarrow \dots \rightarrow v_{k_c}^c.$$

`generate_protocol_TA.py` builds timed automaton per component, using a single local clock x_{r_c} shared by the whole chain and reset at every step, so that it always measures the dwell time elapsed since the token last moved.

Formal Definition: Protocol automaton fragment for a component

Let $\mathcal{G}_c$ be a component with chain $v_1^c, \dots, v_{k_c}^c$ rooted at $r_c = v_1^c$, and let x_{r_c} be its local clock. For each $1 \leq \ell \leq k_c$, a location $N_{v_\ell^c}$ is created, together with one additional absorbing location $\text{Done}_{v_{k_c}^c}$. For $1 \leq \ell < k_c$, a transition

$$N_{v_\ell^c} \xrightarrow[\text{bool}_{v_\ell^c} := \text{false}, \text{bool}_{v_{\ell+1}^c} := \text{true}, \text{total_cost} += \gamma(v_\ell^c), x_{r_c} := 0]{x_{r_c} \geq \delta(v_\ell^c)} N_{v_{\ell+1}^c}$$

advances the token from v_ℓ^c to its successor once the dwell time $\delta(v_\ell^c)$ (chapter 7) has elapsed. The last step of the chain is treated separately: since $v_{k_c}^c$ has no successor, its transition

$$N_{v_{k_c}^c} \xrightarrow[\text{bool}_{v_{k_c}^c} := \text{false}, \text{total_cost} += \gamma(v_{k_c}^c)]{x_{r_c} \geq \delta(v_{k_c}^c)} \text{Done}_{v_{k_c}^c}$$

neither sets any further bool variable nor resets x_{r_c} , since the component has been fully traversed. For each component $\mathcal{G}_c$, the protocol automaton is the tuple

$$\mathcal{A}_c^{\text{proto}} = (L_c, N_{r_c}, \{x_{r_c}\}, \text{Var}_c, \emptyset, E_c, \text{Inv}_c)$$

where $L_c = \{N_{v_\ell^c} \mid 1 \leq \ell \leq k_c\} \cup \{\text{Done}_{v_{k_c}^c}\}$, $\text{Var}_c = \{\text{total_cost}\} \cup \{\text{bool}_{v_\ell^c} \mid 1 \leq \ell \leq k_c\}$, E_c gathers the k_c transitions described above, and $\text{Inv}_c \equiv \text{true}$.

To understand this definition notice the locations $N_{v_\ell^c}$ in the timed automaton correspond to the steps in the protocol execution graph illustrated in Figure 7.2. The described transitions embody what it means to transition from step to another.

Complete automaton

Formal Definition: Protocol automaton $\mathcal{A}_c^{\text{proto}}$ and final composition

The complete Protocol Automaton, serialized in `<file>_protocol.xml`, is the parallel composition

$$\mathcal{A}_{\text{proto}}^+ = \bigotimes_{c=1}^n \mathcal{A}_c^{\text{proto}},$$

Since the clocks $(x_{r_c})_c$ are pairwise disjoint by construction (one per component), only the cost variable `total_cost` and the presence flags $(\text{bool}_i)_{i \in V}$ are shared between the components $\mathcal{A}_c^{\text{proto}}$ — and, more broadly, with the Attack Automaton (subsection 7.2.2), which consumes $(\text{bool}_i)_{i \in V}$ as a guard and also writes to `total_cost` (subsection 7.1.5).

7.2.2 Generation of the attack automaton

Unlike the Property Automaton (subsection 7.2.3), which only *consumes* the variables $(\text{bool}_{i,j})_{i,j}$, the Attack Automaton is the only component of the framework that *produces* them: it is the one that models, for each attack step $\omega \in \Omega$, the actual attempt to carry out this step — with its delay, its cost, and its probabilistic outcome — and that sets $\text{bool}_{\iota(\omega), \sigma(\omega)}$ to true upon success. The script `generate_attack_TA.py` (Figure 7.5) builds this automaton directly from the metrics file μ , without depending on the resolved trees.

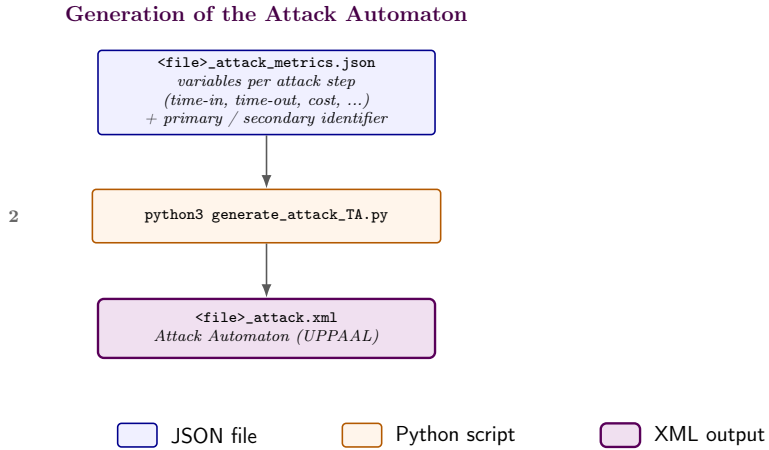


Figure 7.5: **Generation of the Attack Automaton.** The script `generate_attack_TA.py` combines the metrics from `<file>_attack_metrics.json` to produce the final timed automaton `<file>_attack.xml`, directly usable in UPPAAL.

Attack Automata Construction and Composition

For each attack step $\omega \in \Omega_i$, let $\mu(\omega) = (\iota(\omega) \sigma(\omega), w_s(\omega), w_f(\omega), c(\omega), \tau(\omega))$, be its attack metrics.

Formal Definition: Local attack automaton $\mathcal{A}_i^{\text{atk}}$ and final composition

For each primary identifier $i \in V$ such that $\Omega_i \neq \emptyset$, we construct a local timed automaton $\mathcal{A}_i^{\text{atk}}$ that models the execution of its associated attack steps.

To structurally guarantee that the attacker can only attempt one variant of i at a given time, all variants $\omega \in \Omega_i$ originate from a shared, mutually exclusive initial location Idle_i . Upon execution, each variant applies its metrics $\mu(\omega)$ by increasing the global variable `total_cost` and consuming time according to a local clock x_i .

The complete Attack Automaton, serialized in `<file>_attack.xml`, is the parallel composition of these local automata:

$$\mathcal{A}_{\text{attack}}^+ = \bigotimes_{\substack{i \in V \\ \Omega_i \neq \emptyset}} \mathcal{A}_i^{\text{atk}}$$

While the clocks $(x_i)_i$ are pairwise disjoint and local to each $\mathcal{A}_i^{\text{atk}}$, the parallel components globally share the `total_cost` variable and the $(\text{bool}_{i,j})_{i,j}$ success flags. These boolean flags are precisely the variables evaluated by the Property Automaton (subsection 7.2.3) in its violation guards $(\Gamma_p)_{p \in \mathcal{Q}}$.

7.2.3 Generation of the property automaton

The script `generate_result_systems.py`, illustrated in Figure 7.6, takes as input both the resolved attack trees (`<file>_resolved_trees/`) and the metrics file μ produced in the previous section (`<file>_attack_metrics.json`), in order to determine, for each studied security property, the exact condition on the variables $(\text{bool}_{i,j})_{i,j}$ under which it must be considered violated.

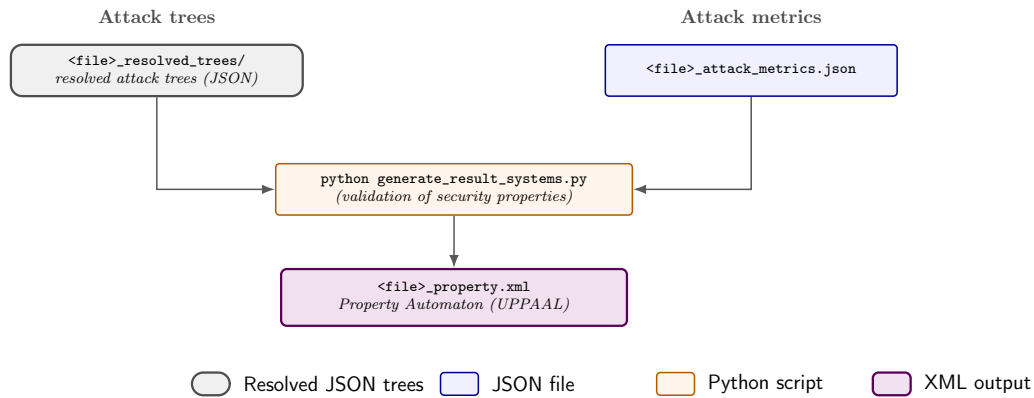
Property Automaton Generation

Figure 7.6: Generation of the Property Automaton. The script `generate_result_systems.py` combines the resolved attack trees (`<file>_resolved_trees/`) and the identified attack metrics (`<file>_attack_metrics.json`) to produce the timed automaton `<file>_property.xml`, which allows checking whether an attack against a security property succeeded.

Grouping trees by property

Several resolved trees $T_k, T_{k'}$ (refer to chapter 7, for a definition of resolved derivation trees) may correspond to the *same* ProVerif query: they then represent *distinct* attack paths against a single security property, obtained via Randomized Proverif when executing the Attack Tree Generation.

Formal Definition: Properties and grouping of trees

Let $\mathcal{Q}$ be the set of *distinct* queried security properties appearing among the processed ProVerif queries (one `not attacker(...)` query per property). We define the map

$$\pi : \{1, \dots, m\} \longrightarrow \mathcal{Q}$$

associating to each index k of resolved tree T_k the property $\pi(k) \in \mathcal{Q}$ that it attacks. For $q \in \mathcal{Q}$, we denote

$$K_q = \pi^{-1}(q) = \{k \in \{1, \dots, m\} \mid \pi(k) = q\}$$

the set of indices of the trees attacking q .

Violation guard: disjunctive normal form

For a queried property p to be violated, it suffices that *at least one* of the attack paths targeting it be fully realized — that is, that all the steps of a *single* tree T_k ($k \in K_p$) have individually succeeded.

Formal Definition: Violation guard Γ_p

For $k \in \{1, \dots, m\}$, let $\varphi_k : N_k \setminus \{r_k\} \rightarrow \Omega$ be the restriction to T_k of the map $n \mapsto \omega(n)$ sending each node to its corresponding attack step ([chapter 7](#)). For each tree T_k , we define the *realization conjunction*

$$C_k = \bigwedge_{n \in N_k \setminus \{r_k\}} \text{bool}_{\iota(n), \sigma(\varphi_k(n))},$$

i.e. the conjunction of the boolean variables associated with each step of T_k (the $\text{bool}_{i,j}$ being those introduced in [chapter 7](#) from μ). The *violation guard* of property $p \in \mathcal{Q}$ is then

$$\Gamma_p = \bigvee_{k \in K_p} C_k,$$

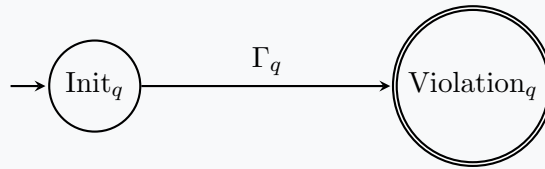
a *disjunctive normal form* (a disjunction of conjunctions), comprising one conjunction C_k per known attack path against p .

Formal Definition: Property automaton $\mathcal{A}_q^{\text{prop}}$

For each property $q \in \mathcal{Q}$, the property automaton is a minimal, two-state timed automaton defined by the tuple:

$$\mathcal{A}_q^{\text{prop}} = (\{\text{Init}_q, \text{Violation}_q\}, \text{Init}_q, \emptyset, \{(\text{bool}_{i,j})_{i,j}\}, \emptyset, \{e_q\}, \text{Inv}_q)$$

Its behavior is governed by a **single transition** e_q connecting the initial state to the violation state, visually represented as follows:



Formally, the transition is $e_q = (\text{Init}_q, \Gamma_q, \varepsilon, \emptyset, \text{Violation}_q)$, where: **Violation guard:** The transition is conditioned by Γ_q .

The state Violation_q is reached if and only if Γ_q is satisfied by the current valuation of the $(\text{bool}_{i,j})_{i,j}$, i.e. if and only if a complete attack path against q has been realized.

Formal Definition: Property automaton $\mathcal{A}_p^{\text{prop}}$

The complete Property Automaton, serialized in `<file>_property.xml`, is the parallel composition

$$\mathcal{A}_{\text{prop}}^+ = \bigotimes_{q \in \mathcal{Q}} \mathcal{A}_q^{\text{prop}},$$

the components sharing the set of variables $(\text{bool}_{i,j})_{i,j}$ declared only once, consistently with `<file>_attack.xml`.

7.2.4 Generation of final automaton

This composition mechanism is exactly the one already informally described in [Figure 7.1](#) at the beginning of the chapter. Now that we have the three automata $\mathcal{A}_{\text{proto}}^+$, $\mathcal{A}_{\text{attack}}^+$ and $\mathcal{A}_{\text{prop}}^+$, the script `generate_final_system.py` computes exactly

$$\mathcal{A}_{\text{final}}^+ = \mathcal{A}_{\text{proto}}^+ \otimes \mathcal{A}_{\text{attack}}^+ \otimes \mathcal{A}_{\text{prop}}^+,$$

serialized in `<file>_final.xml`.

7.3 Implementation

7.3.1 Trying out the implementation

We managed to write a simple implementation. However, because some parts of the framework are not finished, and because the implementation was unstable, we have chosen not to share it here. However we report back with the results of some of the experiments we ran on this toy implementation.

7.3.2 Experimenting with the implementation

We managed to generate a system from the toy example from [subsection 3.2.3](#) and then we check some very basic properties such as execution of the protocol graph or execution of an attack step. This demonstrates that the framework is a promising step toward a sound and functional temporal logic validation. However the problem of combinatorial state explosion soon manifested itself.

7.4 Future Improvements

7.4.1 Implementing the tool into a sound, functional, and user-friendly artefact

Once the framework is formally defined, we call upon future work to implement it as a functional tool.

It is necessary to take a step back and state what we are doing. We are generating an attacker model using Randomize Proverif, then we visualize it using the Tree Generation, and then we analyze the security properties of the protocol for the generated and incomplete attacker. In a way, the generated attacker is an abstraction of the Dolev-Yao attacker.

Modeling conditions, represented with the atomic action using `if then else`, in the protocol execution graph and furthermore in the protocol automaton, is tricky.

The current framework is not easy to understand. However, with further reflection it would be possible to render it user-friendly. Perhaps removing the metrics file for the attacks and rely more on a better protocol execution graph and protocol automaton would make the tool much more easy to use.

7.4.2 Outlining how to solve the combinatorial state explosion system

It soon becomes obvious that combinatorial state explosion is a major bottleneck. We outline two ways of going towards solving it.

Recall the equation :

$$\mathcal{A}_{\text{final}}^+ = \mathcal{A}_{\text{proto}}^+ \otimes \mathcal{A}_{\text{attack}}^+ \otimes \mathcal{A}_{\text{prop}}^+,$$

$\mathcal{A}_{\text{prop}}^+$ is extremely small. In that regard it should not be changed. However we can improve $\mathcal{A}_{\text{proto}}^+$ and $\mathcal{A}_{\text{attack}}^+$.

To reduce the number of states in $\mathcal{A}_{\text{proto}}^+$

$\mathcal{A}_{\text{proto}}^+$ has one state for each atomic action in the applied pi-calculus system. However the atomic action we mostly care about are inputs and outputs, mainly **out** and **in**. In that regard, there is a way to reduce the number of states by removing intermediate atomic actions that are invisible to the attacker (such as **let** bindings), and transferring their associated metrics/data to the remaining visible actions.

To merge $\mathcal{A}_{\text{proto}}^+$ with $\mathcal{A}_{\text{attack}}^+$

There is a way to merge $\mathcal{A}_{\text{proto}}^+$ with $\mathcal{A}_{\text{attack}}^+$. Specifically we can introduce an intermediary step in the framework where we define an automaton $\mathcal{A}_{\text{proto} \& \text{attack}}^+$ whose formal definition is parametrized by $\mathcal{A}_{\text{proto}}^+$ and $\mathcal{A}_{\text{attack}}^+$, in such a way so that $\mathcal{A}_{\text{proto} \& \text{attack}}^+$ is functionally indistinguishable from $\mathcal{A}_{\text{proto}}^+ \otimes \mathcal{A}_{\text{attack}}^+$, and smaller.

Then after the modification the final automaton should become :

$$\mathcal{A}_{\text{final}}^+ = \mathcal{A}_{\text{proto} \& \text{attack}}^+ \otimes \mathcal{A}_{\text{prop}}^+,$$

Chapter 8

Conclusions

8.1 Results

We have made progress toward answering most of the questions raised in the introduction. We have managed to add more nuance to the output of an automated formal method tool in the symbolic model, namely ProVerif, by determining the cost required to carry out an attack. This was realized by writing down the mathematical background of CompareVerif in [section 4.4](#). We have shown a technique to generate more attacks with Proverif in [chapter 5](#). Moreover we have outlined how ProVerif’s output Horn Derivation Trees can be transformed to better abstract the relationship between the attack and the protocol in [chapter 6](#). These new trees can then be further transformed to be analyzed by UPPAAL. Along with data given by the user, the framework in [chapter 7](#) can be used to finalize a tool that quantitatively analyzes the defense against the attack. After a few weeks of additional research, Moving Target Defense could further be represented in the model. We also have shown how modifying Proverif’s source code can help in exhaustive generation of attacks.

Beyond theoretical contributions, this thesis also advances practical system design. Rather than treating theory and practice in isolation, we tightly couple the formal definitions of our algorithms with their concrete implementations. Maintainability is a core design principle: for instance, the modular, three-part parallel composition illustrated in [Figure 7.1](#) ensures that the framework remains intuitive, maintainable, and extensible for industrial engineers.

8.2 Future Works

8.2.1 Related to CompareVerif

Because CompareVerif, built upon ProVerif, is a new tool that implements a paradigm of nuanced bruteforce over ProVerif, there are no known applications besides the ones presented in this document, so far. The bruteforcing in CompareVerif does not necessarily give rise to combinatorial explosion besides perhaps combinatorial explosion related to ProVerif, because the user controls the number of files generated for the analysis. In that regard, CompareVerif could well be used to have nuanced analysis, in the symbolic model, of cryptographic models. The application on a simple version of a ratchet protocol showcases that the paradigm of bruteforcing capabilities in a repetitive protocol like a ratchet could potentially work very well to have information about the minimal cost of an attack. Future works would be about applying this bruteforce mechanism of CompareVerif to the double ratchet of Signal.

8.2.2 Related to Randomized Proverif

Currently, the implementation of the randomization feature remains limited in scope. The primary objective of Randomized ProVerif is to evaluate how modifications to the Horn clause resolution algorithm impact the attack traces discovered by ProVerif. As discussed in [section 5.3](#), there are several potential strategies to achieve this. Beyond simply permuting the list of Horn clauses, another approach could involve investigating the effects of selectively omitting specific clauses. Ultimately, the goal would

be to analyze how combining these two operations—permutation and omission—and applying them multiple times in the resolution algorithm, and not just once at the beginning, might further influence and potentially improve the resulting attack generation by rendering more attacks.

8.2.3 Related to the Attack Tree Generation

Although the tree generation process described in [chapter 6](#) is effective in practice, a formal proof or an informal explanation of its soundness—specifically concerning the abstraction of protocol sequentiality—is yet to be expressed. Furthermore, the parser’s compatibility with rare ProVerif constructs, such as dictionaries or tables, has yet to be fully evaluated.

Unfortunately, in the new tree generation, the thinking process and the computation of the attacker are poorly represented because we have abstracted this information into something relatable to the protocol. Perhaps it is possible to improve the resulting attack tree as to add more information about the computations of the attacker in the resulting tree without sacrificing readability. It is possible to make this more user-friendly.

Implementing the changes described in [subsection 8.2.2](#) and [subsection 8.2.3](#) would further give more answers for problems stated in the introduction in [section 1.1](#), and would give enough material for a conference paper publication with two artefacts combined into one which would be added harmoniously to ProVerif’s backend. It could come with an application on communication systems like the singularisation system described in [\[9\]](#).

8.2.4 Related to Timed Proverif

We have shown that pipelining ProVerif to UPPAAL is possible. However, right now, the outlined algorithm and its implementation is still new. Among the many future works would be to debug the implementation and make it user-friendly. Additionally, we have yet to try to formulate properties in the resulting model. So future works would be about further trying out this new pipeline tool. Among the things to improve :

- Identify to what extent the problem of combinatorial state explosion in the resulting automaton can be solved.
- Seek bugs and debug the existing tool. Seek logical inconsistencies in the definition of the framework and correct them.
- Augment the model pipeline as to add other forms of Mitigation Defense such as Moving Target Defense (MTDs).
- Identify the underlying formal logic of the properties we can check with this, and how it relates to the formal logic behind ProVerif’s Applied Pi-Calculus. Identify which version of UPPAAL we want to use, which type of extended timed automaton is necessary for the underlying formal logic of the tool.
- After exhaustive improvement of the tool, communicate about it at a conference and share it to the community.
- Seek possible applications to an already existing protocol or communication system like the one in [\[9\]](#). Have discussions with industrial partners, in particular, Orange, to assess to what extent such tool could be used to efficiently derive and prove formal security policies in telecommunication contexts.

8.2.5 Abstractions for EFSMs

Unrelated to anything which was discussed above, carried out during the first month of the internship, abstractions to analyze the indistinguishability problem in Extended Finite State Machines (EFSMs) have been discovered. These new abstractions complete the abstractions in [\[10\]](#) in many ways. However because of time constraints these abstractions have not yet been investigated in depth. EFSMs could

potentially be used to model many things in industrial real-life situations, and the three new abstractions could potentially improve the analysis of security properties on these models. Then exhaustive test suites could be implemented to test and analyze (protocol) implementations. Benchmarking could be realized to measure the difference between the abstractions.

Bibliography

- [1] Alur, R., Dill, D.L.: A theory of timed automata. *Theoretical computer science* **126**(2), 183–235 (1994)
- [2] Ballot, G., Malvone, V., Leneutre, J., Borde, E.: Reasoning about moving target defense in attack modeling formalisms. In: *Proceedings of the 9th ACM Workshop on Moving Target Defense*. p. 55–65. MTD’22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3560828.3564009>, <https://doi.org/10.1145/3560828.3564009>
- [3] Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: *Formal Methods for the Design of Real-Time Systems: International School on Formal Methods for the Design of Computer, Communication and Software Systems, SFM-RT 2004, Bertinoro, Italy, September 13-18, 2004, Revised Lectures*. pp. 200–236. Springer (2004)
- [4] Bisping, B., Schlehuber-Caissier, P., Matei, N.: CompareVerif: ProVerif batch analysis for protocol comparison (2026), <https://github.com/benkeks/compareverif>, GitHub Repository
- [5] Blanchet, B.: Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security* **1**(1–2), 1–135 (2016). <https://doi.org/10.1561/33000000004>
- [6] Blanchet, B.: The security protocol verifier proverif and its horn clause resolution algorithm. *Electronic Proceedings in Theoretical Computer Science* **373**, 14–22 (2022). <https://doi.org/https://doi.org/10.4204/eptcs.373.2>
- [7] Blanchet, B., Smyth, B., Cheval, V., Sylvestre, M.: Proverif 2.05: Automatic cryptographic protocol verifier, user manual and tutorial. <https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf> (2025)
- [8] Boureanu, I., Onete, C., Wesemeyer, S., Robert, L., Miller, R., Lafourcade, P., Rajaona, F.: Post-compromise security with application-level key-controls – with a comprehensive study of the 5G AKMA protocol. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. p. 231–247. ASIA CCS ’25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3708821.3733910>
- [9] David, S., Plesa, M., Irimia, S., Poenaru, R.: Privacy-preserving service for secure storage of passwords based on singularization. In: Dumitrache, I., Moisescu, M.A., Pop, F., Copot, D., Popescu, D. (eds.) *25th International Conference on Control Systems and Computer Science, CSCS 2025, Bucharest, Romania, May 27-30, 2025*. pp. 558–562. IEEE (2025). <https://doi.org/10.1109/CSCS66924.2025.00088>, <https://doi.org/10.1109/CSCS66924.2025.00088>
- [10] El-Fakih, K., Kolomeez, A., Prokopenko, S., Yevtushenko, N.V.: Extended finite state machine based test derivation driven by user defined faults. *2008 1st International Conference on Software Testing, Verification, and Validation* pp. 308–317 (2008), <https://api.semanticscholar.org/CorpusID:17305801>
- [11] Jacomme, C., Kremer, S.: An extensive formal analysis of multi-factor authentication protocols. *ACM Trans. Priv. Secur.* **24**(2) (Jan 2021). <https://doi.org/10.1145/3440712>

- [12] Kumar, R., Ruijters, E., Stoelinga, M.: Quantitative attack tree analysis via priced timed automata. In: Sankaranarayanan, S., Vicario, E. (eds.) *Formal Modeling and Analysis of Timed Systems - 13th International Conference, FORMATS 2015, Madrid, Spain, September 2-4, 2015, Proceedings*. Lecture Notes in Computer Science, vol. 9268, pp. 156–171. Springer (2015). https://doi.org/10.1007/978-3-319-22975-1_11
- [13] Larsen, K.G., Pettersson, P., Yi, W.: Uppaal in a nutshell. *International Journal on Software Tools for Technology Transfer* **1**(1-2), 134–152 (1997)
- [14] Lowe, G.: An attack on the Needham–Schroeder public-key authentication protocol. *Information Processing Letters* **56**(3), 131–133 (1995). [https://doi.org/10.1016/0020-0190\(95\)00144-2](https://doi.org/10.1016/0020-0190(95)00144-2)
- [15] Matei, N., Bispin, B., Schlehuber-Caissier, P., Kushik, N., Gaber, C., Wary, J.P.: Orange–Télécom SudParis collaboration: Formal verification of security properties in moving target defense strategies. In: *Approches Formelles pour l’Assistance au Développement de Logiciels (AFADL)* (2026)
- [16] Meier, S., Schmidt, B., Cremers, C., Basin, D.: The TAMARIN prover for the symbolic analysis of security protocols. In: Sharygina, N., Veith, H. (eds.) *Computer Aided Verification*. pp. 696–701. Springer Berlin Heidelberg, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_48
- [17] Mendes, N., Madeira, H., Duraes, J.: Security benchmarks for web serving systems. In: *2014 IEEE 25th International Symposium on Software Reliability Engineering*. pp. 1–12 (2014). <https://doi.org/10.1109/ISSRE.2014.38>
- [18] Moser, F.: VSCode ProVerif language service (2026), <https://github.com/ProVerif/vscode-proverif-language-service/>, GitHub Repository
- [19] Nicoletti, S.M., Lopuhaä-Zwakenberg, M., Hahn, E.M., Stoelinga, M.: ATM: A logic for quantitative security properties on attack trees. In: Ferreira, C., Willemse, T.A.C. (eds.) *Software Engineering and Formal Methods*. pp. 205–225. Springer Nature Switzerland, Cham (2023). <https://doi.org/10.1007/s10270-025-01323-z>
- [20] Oliveira, R.A., Raga, M.M., Laranjeiro, N., Vieira, M.: An approach for benchmarking the security of web service frameworks. *Future Generation Computer Systems* **110**, 833–848 (2020). <https://doi.org/10.1016/j.future.2019.10.027>
- [21] Paier, M., Van Eeden, R., Miculan, M.: Formal analysis of multi-factor authentication schemes in digital identity cards. In: Madeira, A., Knapp, A. (eds.) *Software Engineering and Formal Methods*. pp. 423–440. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-77382-2_24
- [22] Schneier, B.: Attack trees. <https://www.schneier.com/wp-content/uploads/2015/12/attacktrees.pdf> (1999)

Automated Comparison of Protocol Security using Symbolic Verification*

Benjamin Bisping ¹, Nicolas Matei¹, Natalia Kushik ¹,
Philipp Schlehuber-Caissier ¹, Chrystel Gaber², and Jean-Philippe Wary²

¹ SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, Palaiseau, France

² Orange Innovation, Paris, France

Abstract. We investigate how to compare the security of protocols in terms of their resilience through symbolic verification. As a theoretical framework, we use hierarchies of attack capability combinations, which we search for cheapest ways to break security properties. For the practical implementation, we use the ProVerif verifier to process families of models parameterized over priced attacker capabilities. Our tooling improves the convenience of security benchmarking through attacker capabilities, Pareto fronts of attack budgets, and attack trees. As a case study, we examine how classical authentication protocols with hashed passwords can be hardened through singularization, a mechanism to transparently upgrade (weak) secrets in communication systems.

Keywords: Protocol security · Symbolic verification · Quantitative security comparison · ProVerif · Attack trees · Singularization

1 Introduction

The comparison of communication systems with respect to their security is a recurring topic. Generally, the problem is stated as follows: given two (or more) functionally equivalent implementations, which one is *more secure*? While there is a theoretical perspective on such questions in terms of computational capabilities of an attacker, the engineering perspective rather cares about system resilience: how many things need to go wrong for the system to be compromised? The answer is often expressed through attack trees [20] and vulnerability comparison [15]. Combined with security metrics [17], such an analysis allows one to name a total price for a given attack scenario and thus provides a quantitative measurement of system security with respect to a given attack model.

Deriving an exhaustive attack tree is difficult, but it can be solved through the use of security checking tools. However, symbolic protocol verifiers such as ProVerif [3] and Tamarin [14] usually check if given security properties hold for a system, only leading to *Yes* or *No* replies without quantitative analysis. Moreover, these provers work on an *individual* system with a fixed threat scenario.

* The work is supported by the bilateral Orange and Télécom SudParis CRE Project “Formal validation of the singularization principle.”

2 B. Bisping et al.

But for comparisons, one has to pair *multiple* protocols with ranges of threat models. So, how can symbolic verification be used to compare different threat models, prices, and security metrics in a meaningful way?

The state of the art is to either check different protocol variants by hand, commenting and uncommenting parts of the model, or to use existing preprocessors to enumerate protocol variants. Then a checker like ProVerif is run and the information is accumulated mostly manually, to interpret the needed capabilities of an attacker [7,10,19]. Implicitly, such work assumes different attacker capabilities and scenarios. But it does not make these an explicit part of the process, where one can, for instance, link it to reasoning about prices in attack trees. This paper fills this gap, proposing a general methodology and a tool to conveniently compare protocol security with symbolic verifiers such as ProVerif.

Contributions. This paper presents theory, implementation, and a case study of how to compare security properties of systems using automatic verification:

- Section 3 describes a *mathematical model* for reasoning about hierarchies of attack models, parameterized in terms of combinations of priced capabilities, together with the methodology for a quantitative assessment of security for communication protocols.
- We automate the proposed approach using the symbolic verifier ProVerif in Section 4. Our tool *CompareVerif* [2] enumerates attack scenarios specified in a single file and uses ProVerif to automatically measure security in a quantitative fashion. It can also provide the augmented attack graphs and Pareto fronts of attack budgets for user convenience.
- With Section 5, we show the applicability of the proposed approach by formalizing the security arguments of David et al. [8] to compare a *singularized* password storage and authentication protocol with a baseline variant.

To introduce the background, Section 2 illustrates the problem of modeling systems and attack capabilities in ProVerif. Section 6 connects the paper to related work in protocol analysis and security benchmarking.

2 Background

In this section, we sketch how to examine the security of a protocol in the symbolic model. We will go through the basic concepts along a ProVerif example to set the stage for their abstract treatment in Section 3 and our implementation in Section 4. The underlying insight is that users applying such security verifiers do not only care about the prover returning true-or-false for their model, but also about checking what needs to go wrong for attacks to be successful.

Symbolic protocol verifiers check for properties on models of protocols that are equipped with an algebraic theory of how messages can be transformed. There exist several mature tools providing automatic solutions for this task. For the scope of this work, we use ProVerif [3], which over-approximates attacks in the Dolev–Yao model through a constraint system. Intuitively, the

attacker has control over all exposed channels and is therefore able to intercept, read, block, delete, inject, modify, and replay messages. A typical class of vulnerabilities to be uncovered like this are man-in-the-middle attacks like on the Needham–Schroeder protocol [12]. ProVerif checks whether attacks found in the over-approximation can be replayed on the concrete protocol model. Probabilistic and asymptotic aspects of cryptography, such as computational indistinguishability, are not represented in ProVerif.

2.1 Example: Hashed Password Storage and Authentication

In the following, we illustrate our points using a simple password authentication and storage protocol. The running example handles user passwords in hashed form as follows: A client sends the credentials, and the server checks if such a user can be authenticated, i.e. if the user id and hashed password are in the user database. If this is the case, access is granted. The sent messages are encrypted, and for the sake of simplicity, the client has an unlimited number of authentication attempts. The ProVerif model for the example is available in `examples/hashed_passwords_paper.pv` of the CompareVerif repository [2].

Equational theories in ProVerif. The theory of public-key cryptography can be expressed as:

```
type seckey.
type pubkey.

fun pk(seckey): pubkey.
fun aenc(bitstring, pubkey): bitstring.
reduc forall m: bitstring, sk: seckey; adec(aenc(m, pk(sk)), sk) = m.
```

This snippet declares two types `seckey` and `pubkey` to describe secret and public keys, as well as two constructor functions: `pk` to generate the public key for a given secret key and `aenc` to encode messages with `bitstring` payloads. The `reduc` rule states that an original message `m` can be retrieved by `adec` if one possesses the secret key `sk` that gave rise to a public key `pk(sk)`. This equation creates opportunities for the system and the attacker alike.

In the absence of equations, `fun`-functions are understood to be strict one-way functions whose values cannot be constructed any other way. So, perfect hashing is expressed simply by declaring a function (with two inputs for payload and salt, both given as bitstrings) without equations:

```
fun hashed(bitstring, bitstring): bitstring.
```

Specifying systems in ProVerif. Systems in ProVerif are modeled in the applied π -calculus, with a number of convenience features, e.g., to declare `tables`, modeling values stored in databases. In the example, we have a table of password entries consisting of user ids `uid` together with hashed passwords and salts as `bitstrings`:

```
type uid.
table passwd(uid, bitstring, bitstring).
```

4 B. Bisping et al.

Communication is modeled through channels that can be filled with messages by out-statements and read from by in-statements. Also, there are expressions to read from databases `get...suchthat` and to mark specific points in the execution flow by `events`. In the example, the server process reads authentication requests from a channel `server_link`, decodes them using its `server_secret` key, and checks them against hashed salted passwords in the `passwd` table:

```
channel server_link.
event server_user_authenticated(uid).

let server_app(server_secret: seckey) =
  in (server_link, ct: bitstring);
  let (u: uid, password: bitstring) = adec(ct, server_secret) in
  get passwd(uid1, hashedpw: bitstring, salt: bitstring)
    suchthat uid1 = u && hashedpw = hashed(password, salt) in
    event server_user_authenticated(u).
```

Here, `event server_user_authenticated(u)` marks successful authentication.

Following the example, we create an example user, store their hashed secret by `insert passwd(user1, hashed(pw1, salt1), salt1)`, create a fresh new private key `server_secret`, and use it to send encrypted authentication requests for `user1` with `pw1` to `server_link`.

```
free user1: uid.
free pw1: bitstring [ private ].
free salt1: bitstring [ private ].

process
  insert passwd(user1, hashed(pw1, salt1), salt1);
  new server_secret: seckey;
  ( !server_app(server_secret)
    | !out (server_link, aenc( (user1, pw1), pk(server_secret) )))
```

The `!` symbols in the parallel composition `|` represent replication, meaning that we do not restrict how often these things may occur (in parallel or in sequence).

2.2 Checking Protocol Security

So far, the example protocol is secure in the sense that a Dolev–Yao attacker cannot learn about the password or its hashed form. In ProVerif, this is expressed by queries, and in the example we will only use their simplest variant:

```
(* no pw leakage *)
query attacker(pw1).
(* no hashed pw leakage *)
query attacker(hashed(pw1, salt1)).
```

Here, `attacker(pw1)` triggers ProVerif to check for ways for the attacker to learn `pw1`. In this instance, it establishes the absence of successful attacks for both properties; as mentioned above, the protocol is secure. More generally, such a security check can have four outcomes:

- ✓ Yes, ProVerif can prove that the attacker cannot make the query true (i.e., it checks for the negation of the stated property).
- ✗ No, there is a violation of the query that ProVerif can replay as a specific trace of attacker interactions with the system.
- ? Inconclusive, ProVerif's over-approximation hints at attacks but none could be reconstructed in the model.
- ⊥ ProVerif fails to terminate (in the time one allows it to run).

Similar tools have comparable interfaces, and in the spirit of genericity, upcoming sections will simply assume the presence of a model-checking function

$$\text{check}: \mathcal{IE} \times \mathcal{Q} \rightarrow \{\checkmark, \times, ?, \perp\},$$

where $\mathcal{IE}$ stands for the language of applied π -calculus systems enhanced with an equational theory, to be checked against queries from a language $\mathcal{Q}$.

So, let us call the π -system and equational theory in the running example πe_{hashed} . As ProVerif can prove the secrecy of `pw1`, we have:

$$\text{check}(\pi e_{\text{hashed}}, \text{attacker}(\text{pw1})) = \checkmark.$$

2.3 Combining Attacks to Break Properties

We now consider scenarios when security properties fail. For example, consider a case where, after private key creation, `server_secret` is sent publicly across `server_link` in the main process:

```
out (server_link, server_secret);
```

In the Dolev–Yao model, this means that the attacker can obtain the secret, and after eavesdropping on a user authentication request, use `adec` to decrypt the password. On this modified model $\pi e_{\text{hashed}}^{\text{compromised}}$, we thus have

$$\text{check}(\pi e_{\text{hashed}}^{\text{compromised}}, \text{attacker}(\text{pw1})) = \times.$$

As a scant consolation, the hashed variant remains confidential:

$$\text{check}(\pi e_{\text{hashed}}^{\text{compromised}}, \text{attacker}(\text{hashed}(\text{pw1}, \text{salt1}))) = \checkmark.$$

Another scenario of compromise might be that the database is leaked to the attacker, $\pi e_{\text{hashed}}^{\text{db-leak}}$, expressible by extending the main parallel composition with arbitrary reads from the `passwd` table:

```
| !get passwd(u, hashedpw, salt) in
  out (server_link, (u, hashedpw, salt))
```

This invalidates queries in a fashion inverted to $\pi e_{\text{hashed}}^{\text{compromised}}$, i.e.,

$$\text{check}(\pi e_{\text{hashed}}^{\text{db-leak}}, \text{attacker}(\text{hashed}(\text{pw1}, \text{salt1}))) = \times.$$

As a last scenario, let us consider $\pi e_{\text{hashed}}^{\text{rainbow}}$, where the equational theory of πe_{hashed} is extended by a possibility to undo hashing if one knows the salt, e.g., due to a rainbow table attack or weak passwords:

6 B. Bisping et al.

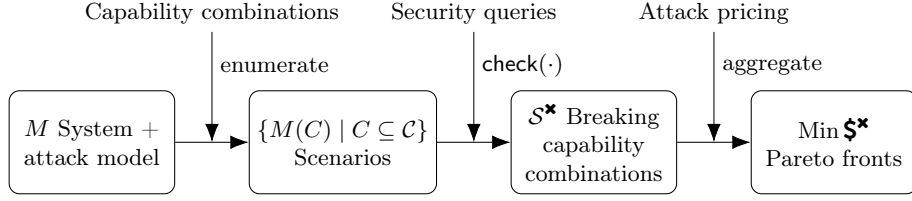


Fig. 1. Sketch of how to derive attacker's Pareto fronts per query for priced attacker capabilities.

```

reduc forall m: bitstring, s: bitstring;
  unhash(hashed(m, s), s) = m.

```

Note that the possibility of unhashing, on its own, does not break any queries. However, in combination with `db-leak`, the attacker could use it to break *both* queries.

When working with different models of threats and compromises, people end up with ProVerif files that list different attacker capabilities that are included or excluded through code comments or preprocessing. In Section 3, we will make explicit the assumptions underlying such a practice.

3 Reasoning about Capabilities

In this section, we provide a mathematical description of the ideas behind the work with hierarchies of attack capabilities. Figure 1 gives an overview: A system and attack model M serves as input to be instantiated to scenarios with different combinations of attack capabilities. For each combination, queries of security properties are checked. The minimal combinations of capabilities to break properties give rise to Pareto fronts of cost-minimal attacks.

3.1 Modeling Attacker Capabilities

Given a set of *attack capability names*, $\mathcal{C} = \{c_1, c_2, \dots\}$, we refer to subsets of capabilities $C \subseteq \mathcal{C}$ as *capability combinations*. As main input we consider a family of models parameterized in terms of capability combinations, that is, a function $M: 2^{\mathcal{C}} \rightarrow \Pi\mathcal{E}$. We call the values $M(C)$ with $C \subseteq \mathcal{C}$ *scenarios*. Each scenario describes a pairing of the system with an attack model, where the baseline (that is $M(\emptyset)$) is the Dolev-Yao model.

Per query $q \in \mathcal{Q}$, we denote by $\mathcal{S}_q^\checkmark$ the set of capability combinations for which the property q holds (i.e., the protocol is secure), and analogously $\mathcal{S}_q^\times$ for the ones where q does not hold:

$$\begin{aligned} \mathcal{S}_{M,q}^\checkmark &:= \{C \subseteq \mathcal{C} \mid \text{check}(M(C), q) = \checkmark\} \\ \mathcal{S}_{M,q}^\times &:= \{C \subseteq \mathcal{C} \mid \text{check}(M(C), q) = \times\} \end{aligned}$$

Generally, we assume that M is defined in such a way that having more attack capabilities in C renders more attacks successful, that is:

$$C \subseteq C' \text{ and } \text{check}(M(C), q) = \mathbf{X} \text{ implies } \text{check}(M(C'), q) = \mathbf{X}$$

and conversely (although with four-valued $\text{check}(\cdot)$, not equivalently):

$$C \subseteq C' \text{ and } \text{check}(M(C'), q) = \mathbf{V} \text{ implies } \text{check}(M(C), q) = \mathbf{V}.$$

Intuitively, this means that attacker capabilities do not hinder one another, and adding a capability never makes a previously successful attack fail. This monotonicity implies that, on the powerset lattice $(2^{\mathcal{C}}, \subseteq)$, we can focus on the minimal attack capability combinations to break queries:

$$\text{Min } \mathcal{S}_{M,q}^{\mathbf{X}} := \{C \in \mathcal{S}_{M,q}^{\mathbf{X}} \mid \nexists C' \in \mathcal{S}_{M,q}^{\mathbf{X}}. C' \subset C\}$$

Families on the authentication example. For our running example of Section 2, we would have the following capabilities in $\mathcal{C}_{\text{hashed}}$:

compromised	Attacker breaks asymmetric encryption of server communication.
db-leak	Attacker can read from the database of hashed passwords.
rainbow	Attacker can undo the hashing if they know the salt.

So, implicitly, we have already constructed the scenarios:

$$\begin{aligned} M_{\text{hashed}}(\emptyset) &= \pi e_{\text{hashed}} \\ M_{\text{hashed}}(\{\text{compromised}\}) &= \pi e_{\text{hashed}}^{\text{compromised}} \\ M_{\text{hashed}}(\{\text{db-leak}\}) &= \pi e_{\text{hashed}}^{\text{db-leak}} \\ M_{\text{hashed}}(\{\text{rainbow}\}) &= \pi e_{\text{hashed}}^{\text{rainbow}} \end{aligned}$$

of the family M_{hashed} in Section 2.3. The capability abstraction allows us to conveniently state our observation about the combinability of leak and rainbow attacks to break user password secrecy, that is:

$$\text{check}(M_{\text{hashed}}(\{\text{db-leak}, \text{rainbow}\}), \text{attacker}(\text{pw1})) = \mathbf{X}.$$

So, we have observed two ways of violating this property, which are incomparable and minimal:

$$\text{Min } \mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^{\mathbf{X}} = \{\{\text{compromised}\}, \{\text{db-leak}, \text{rainbow}\}\}$$

Figure 2 gives an overview of the hierarchy of capability combinations that allow to break the password secrecy.

8 B. Bisping et al.

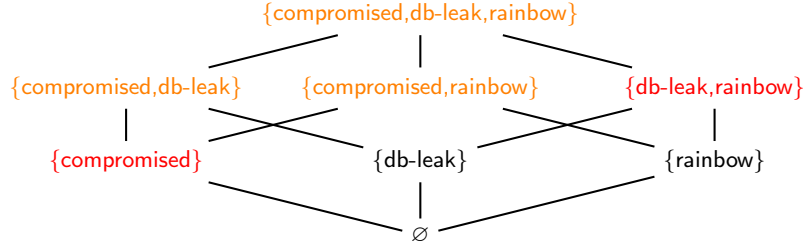


Fig. 2. Hasse diagram of the partially ordered set of capability combinations $(2^{\mathcal{C}}, \subseteq)$ for the example. The subset $\mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^*$ where query **attacker(pw1)** breaks is highlighted, singling out its minimal elements $\text{Min } \mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^*$ in red.

3.2 Pricing Attacker Capabilities

If one wants to compare two protocols, it might not always be the case that there is a one-to-one correspondence between specific attacker capabilities in both models. To compare protocols under such situations, one must translate the capabilities into some shared cost space in order to contrast how hard or resource-intensive the attacks are. Such costs could, for instance, be time and computational resources necessary to brute-force secrets, the need for physical access or user interaction, availability of exploits, or hacking skills. The cost dimension can sometimes be inferred from attack descriptions in well-known databases like CVE or MITRE ATT&CK. But generally, modeling decisions regarding the attacker profile and abstractions are necessary.

Given a *pricing function* to assign costs from $\mathcal{P}$ to attack capability combinations $\text{price}: 2^{\mathcal{C}} \rightarrow \mathcal{P}$, we denote by $\mathcal{P}_i$ the i -th component of $\mathcal{P}$ and we typically use $\mathbb{N}^m$ or $\mathbb{R}^m$ as $\mathcal{P}$ with m being the number of cost dimensions. We say that a capability combination C with price $P = \text{price}(C)$ is *cheaper* than C' with price $P' = \text{price}(C')$, denoted $P \leq P'$, if $P_i \leq P'_i$ for all price dimensions i . Furthermore, we say that C is *strictly cheaper* than C' , denoted $P < P'$, if $P \leq P'$ and there exists at least one price dimension i such that $P_i < P'_i$.

Assuming monotonicity for the pricing function, that is, for all $C, C' \in \mathcal{C}$, $C \subseteq C'$ implies $\text{price}(C) \leq \text{price}(C')$, we can derive the Pareto front of cost-minimal attacks for a query from the minimal capabilities:

$$\begin{aligned}
 \text{Min } \mathcal{S}_{M,q}^* &= \{P \mid P = \text{price}(C), C \in \text{Min } \mathcal{S}_{M,q}^* \text{ and} \\
 &\quad \nexists C' \in \text{Min } \mathcal{S}_{M,q}^*. \text{price}(C') < P\} \\
 \mathcal{S}_{M,q}^* &= \uparrow(\text{Min } \mathcal{S}_{M,q}^*) = \{P \in \mathcal{P} \mid \exists P' \in \text{Min } \mathcal{S}_{M,q}^*. P' \leq P\}
 \end{aligned}$$

We say that a protocol A with model M_A is more secure than protocol B with M_B with respect to a property that is expressed as q_A in A and as q_B in B if the attacker has a smaller space of sufficient budgets:

$$\mathcal{S}_{M_A, q_A}^* \subset \mathcal{S}_{M_B, q_B}^*$$

meaning that any succeeding attack in A with price P_A can be matched by an attack in B with price $P_B \leq P_A$ and there is at least one attack for which the inequality is strict, that is, an attack on protocol B is strictly cheaper.

Prices on the authentication example. Continuing our running example, we use a two-dimensional discrete vector space $\mathcal{P}_{\text{hashed}} := \mathbb{N}^2$ to quantify how much time (first dimension) and hacking effort (second dimension denoted “hack”) an attacker would need.

We define the pricing $\text{price}_{\text{hashed}}: \mathcal{C}_{\text{hashed}} \rightarrow \mathbb{N}^2$ as the vector sum of costs per capability

$$\text{price}_{\text{hashed}}(C) := \sum_{c \in C} p(c)$$

$$\text{where } p(c) := \begin{cases} (0, 4) & c = \text{compromised} \\ (0, 1) & c = \text{db-leak} \\ (2, 0) & c = \text{rainbow} \end{cases}$$

Knowing that $\text{Min } \mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^* = \{\{\text{compromised}\}, \{\text{db-leak}, \text{rainbow}\}\}$, we can compute the minimal attack budgets to break password secrecy:

$$\begin{aligned} \text{Min } \mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^* &= \{p(\text{compromised}), p(\text{db-leak}) + p(\text{rainbow})\} \\ &= \{(0, 4), (0, 1) + (2, 0)\} = \{(0, 4), (2, 1)\} \end{aligned}$$

The minimal attack budgets to break hashed password secrecy boil down to a single point: $\text{Min } \mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{hashed}(\text{pw1}, \text{salt1}))}^* = \{p(\text{db-leak})\} = \{(0, 1)\}$. Figure 3 visualizes the Pareto front of minimal attack budgets for the two queries in the hashed password authentication example. The blue areas represent the sufficient budgets $\mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{pw1})}^*$ and $\mathcal{S}_{M_{\text{hashed}}, \text{attacker}(\text{hashed}(\text{pw1}, \text{salt1}))}^*$, respectively. That the second one is larger than the first one is a visual confirmation of the fact that it is easier to learn the hashed password than to learn the password itself.

4 Tooling to Survey Attacks and Adversary Capabilities in ProVerif

We now turn to the question of how to make it convenient to explore hierarchies of attack capabilities using ProVerif. For this, we have implemented *CompareVerif*, a collection of Python scripts as a ProVerif harness available on GitHub [2].

The basic idea is to enhance ProVerif’s .pv files with certain *magic comments* that describe priced attacker capabilities in the sense of Section 3. Our tooling implements the workflow from Figure 1 and at the same time addresses several pain points:

10 B. Bisping et al.

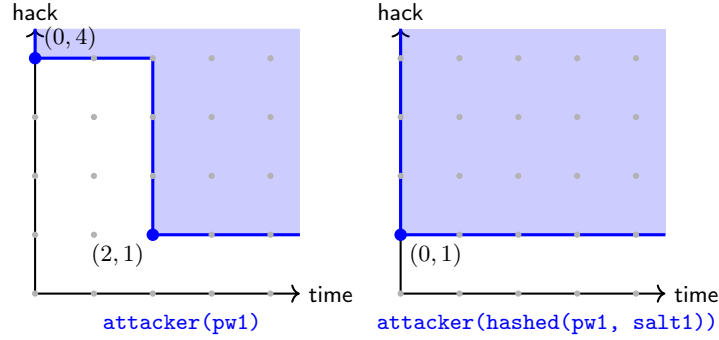


Fig. 3. Pareto fronts of attack budgets for the two queries in the hashed password authentication example.

- CompareVerif *automates the exhaustive handling* of different attack scenarios. On the one hand, this is more robust than manual inclusion and exclusion of compromising model parts, and on the other, our tool can be combined with other automatic workflows.
- The comment approach ensures *maintainability* of input files through *compatibility with IDEs* (in particular, the VS Code plugin [16]) which is not necessarily true for other preprocessor languages.
- We *automate the extraction of query results* from ProVerif’s otherwise overwhelming console output and make it easy to interpret them in terms of an attack cost model.
- We *explain attacks by attack trees linked back to capabilities*, which are more intelligible to engineers than ProVerif’s derivation trees.

4.1 Attack Scenario Batch Processing

To describe the model M , the user specifies the protocol and the equational theory in a ProVerif file. The capabilities are inserted using special comments of the following syntax:

```
(** NameOfCapability [CostOfCapability CostDimensionName]
...
**)
```

For example, the db-leak capability of a compromised database at the cost of one hack would be added to the main process as a comment:

```
process
...
(** DB Leak [1 hack]
| !get passwd(u, hashedpw, salt) in
  out (server_link, (u, hashedpw, salt))
**)
```

Note that queries in ProVerif are anonymous. Therefore, in order to facilitate the interpretation of ProVerif output, the tool extracts comments in front of queries as names. So, for instance, we consider “no pw leakage” in the following snippet as the name of the query.

```
(* no pw leakage *)
query attacker(pw1).
```

Our Python implementation is executed by starting:

```
python scenario_preprocessor.py [location of .pv file]
```

For example, the output for the running example from Section 3 is the following:

```
python scenario_preprocessor.py examples/hashed_passwords_paper.pv
[...]
Query: no pw leakage
      {Database leak, Rainbow table attack} (cost: 1 hack, 2 time)
      {Server compromised} (cost: 4 hack)
Query: no hashed pw leakage
      {Database leak} (cost: 1 hack)
```

The script runs in four stages, aligning with the abstract workflow of Figure 1:

1. Parse the .pv file to extract model M , capabilities $\mathcal{C}$, and pricing p .
2. Generate one .pv scenario file per capability combination and assemble these files together with possible `-lib` dependencies in a `_scenarios` folder.
3. Run ProVerif on each file, collecting the information on query results and prices in a `manifest.json` file.
4. Report the minimal prices and capabilities to break queries to the console.

The script can also take several ProVerif files as arguments to facilitate comparisons, as well as several flags to control execution details. For instance, one can control whether the script should try to only consider scenarios that are not dominated by results that have already been obtained. Otherwise, the script will generate and check $2^{|\mathcal{C}|}$ variants of a model, where $|\mathcal{C}|$ is the number of attacker capabilities. Details are documented in `README.md` in [2]. For the small example in this paper, the script runs instantaneously.

We also provide a script `pareto_comparison.py` that can render Pareto fronts resembling those of Figure 3, from `manifest.json` results.

4.2 Extracting Attack Trees

From an engineering standpoint, it is particularly important to be able to analyze the property violations, found by security provers, as ProVerif. For example, this enables users to validate that the attack model is specified in a reasonable way. And when there are surprising attacks, one wants to understand them to improve the protocol. For that reason, our tool supports the generation of *attack trees*, which are a standard approach to comprehend the attack surface of a system [20].

12 B. Bisping et al.

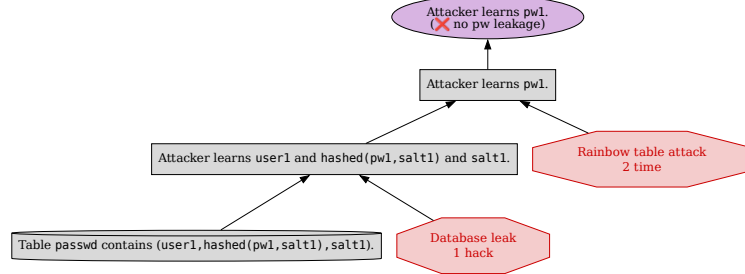


Fig. 4. Attack tree against the password security query for the {db-leak,rainbow}-scenario on the running example.

We provide a script that renders the conjunctive part of an attack graph that gives rise to a specific attack for a ProVerif model, in particular, for our generated scenarios. Coming back to the running example, for the {db-leak,rainbow}-scenario, `attack_tree_extractor.py` returns the tree in Figure 4. The top node names the violated security property, the red leafs represent the capabilities used in the attack, and the gray nodes state intermediate facts or attacker goals.

The attack trees are generated from the intermediate information of ProVerif’s attack derivation system, combined with our scenario information. Under the hood, ProVerif uses a symbolic form of attack trees (or rather graphs) to find potential attacks in a Horn clause derivation system [4]. To generate trees in the form of Figure 4, we first extract the derivation of an attack found by ProVerif. Further, we extract the clause sets from singleton scenarios in order to find out which capabilities give rise to which derivation rules. With this information, we relate the derivations back to the capability snippets to accordingly annotate the trees with capability nodes. Note that when constructing such trees for cost-minimal scenario/query combinations, we can be sure that the tree expresses a cost-minimal and valid attack, because, on the one hand, higher-cost attacks are not present in these scenarios, and, on the other, ProVerif checks attacks for their validity.

The output of our tool only outlines *conjunctive* parts of attack trees, disregarding alternative options an attacker might have to attain the same goals. Such information is omitted; it is present plentifully in ProVerif’s backend, but goes beyond the derivation reconstruction approach. Moreover, it can potentially lead to an overwhelming output. Also, technically, the attacks are not necessarily trees: if capabilities or established facts are used multiple times in the same derivation, the node is reused, leading to a directed acyclic graph structure. Aside from such nuances, the option to extract attack trees as described is one of the great advantages of the symbolic approach to security verification.

5 Case Study: Singularization

As a small case study of how to use the approach and tool to compare system security, we will examine a *singularized* version of password authentication by David et al. [8]. The singularization technique [9] aims to transparently upgrade user secrets by mixing in stronger ones that are handled by a third-party server. The core argument for its usefulness when handling user passwords is that, in the case of an exfiltration of the user database, singularization shields user passwords from offline brute-force attacks.

5.1 Modeling the Singularized Authentication System in ProVerif

David et al. [8] enhance an authentication protocol with a third party that strengthens user passwords through high-entropy secrets stored independently of the main user data. The central twist is that, in the authentication process, the server sends an ElGamal-encrypted version of the user password to a singularization service. Due to the homomorphic encryption property of ElGamal, the singularization service can add a part to the secret without learning the user password itself. The main server can thus obtain the concatenation of user password and singularization part by decrypting the modified message and then check for the presence of the enhanced user password in the database. Our ProVerif model of the encrypted system is in `examples/singularized_passwords_paper.pv`. Modeled in ProVerif, the core change on the server side looks as follows:

```
let server_app(server_secret: seckey, singularization_pk: pubkey) =
  in(server_link, ct: bitstring);
  let (u: uid, password: bitstring) = adec(ct, server_secret) in
  new elgamal_secret: bitstring;
  new answer_channel: channel;
  out(singularization_link,
    aenc(
      (u, egenc(password, elgamal_secret), answer_channel),
      singularization_pk
    ));
  in(answer_channel, singularized_pw_enc: bitstring);
  let singularized_pw = egdec(singularized_pw_enc, elgamal_secret) in
  get passwd(uid1, hashedpw: bitstring, salt: bitstring)
  suchthat uid1 = u && hashedpw = hashed(singularized_pw, salt) in
  event server_user_authenticated(u).
```

5.2 Specifying Attacks and Prices

Naturally, singularization adds new points where one should consider additional attack vectors. At the same time, syntactically equivalent capabilities might have different prices in the new setting.

- The Server compromised attack is an equivalent problem in both settings. Like in Subsection 3.2, we price it at 4 hack, as it requires a more involved online attack on the main server.

14 B. Bisping et al.

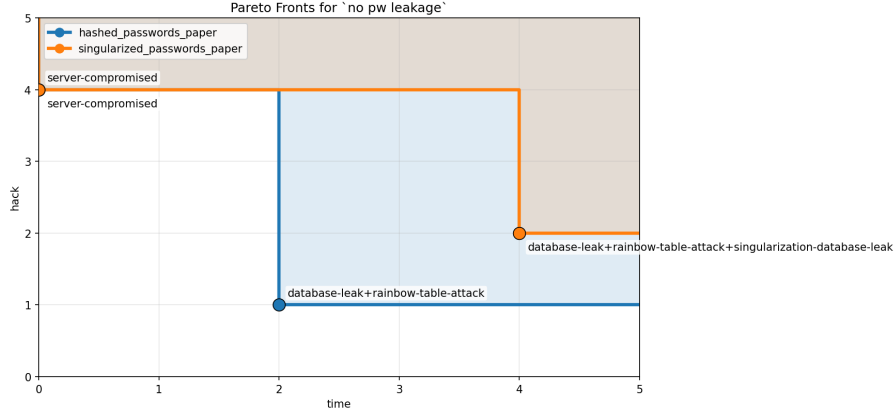


Fig. 5. Pareto front output of CompareVerif comparing password secrecy on models of hashed and singularized handling of authentication.

- Database leak is a less severe case, possible in both cases. Because it can happen through some offline exfiltration (e.g., problems with the backup security), we only assign the price of 1 hack.
- Rainbow table attack also is possible in both protocols. While we set its price at 2 time for the standard hashed version, we double it to 4 time in the singularized version. The reason is that filling a rainbow table primarily is a question of estimated time; and this estimate raises when longer bitstrings have to be hashed, which is the case with added singularization strings.
- Singularization server compromised might only occur in the singularized setting. We give a price of 3 hack, as it would also require an online attack, but the defender might have less control over the singularization service. (In any case, this attack does not open a weakness as long as the ElGamal encryption scheme is safe.)
- Singularization database leak, similarly to the database leak, is priced at 1 hack, but only applies to the singularized system.

5.3 Benchmarking Using CompareVerif

We can now use *CompareVerif* to exhaustively compare our model of standard password authentication under attacks M_{hashed} with the singularized variant developed in this section. A Jupyter notebook for this comparison is available in `notebooks/comparison_notebook_paper_shell.ipynb`.

Figure 5 shows the Pareto front output to compare the security of the `attacker(pw1)` query in hashed (blue) and singularized (orange) password handling. Clearly, an attacker needs higher budgets in terms of hacks and time in order to crack the user password, which showcases the positive impact of the singularization.

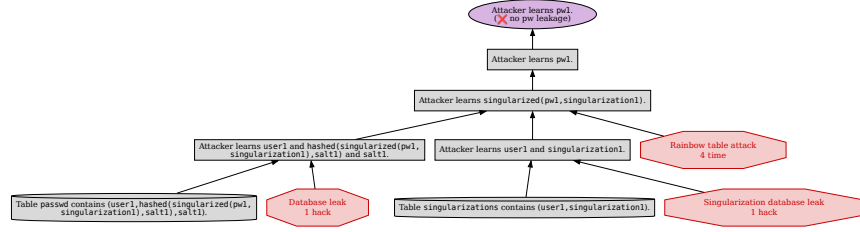


Fig. 6. More complex attack tree against password secrecy for singularized model.

To understand how the higher price comes together, we can examine the attack tree for the second point of the Pareto front, given in Figure 6. Its key difference from Figure 4 to break the hashed scenario is that another hacking step is required to break the singularization database and that the computation of the rainbow table takes more time.

In conclusion, *CompareVerif* supports us in benchmarking models of systems and attacks against each other by reducing the exhaustive thinking of how system features and attacks might come together to ProVerif calls. Our models give a formal shape to the arguments by David et al. [8] about the practical security advantages of singularization.

6 Related Work

Our approach connects to existing work by proposing to combine priced attacker capabilities with automated symbolic verification, aiming to compare security levels of communication protocols.

Attack scenarios in ProVerif. The closest in spirit to the present paper is the work by Jacomme and Kremer [10], who model and enumerate threat scenarios in ProVerif to compare the security of multi-factor authentication mechanisms. Unfortunately, their artifact link is not active; so we cannot report their mechanization in detail. The approach is continued by Paier et al. [19], where threats are described through interfaces of protocol entities and their level of access to them. Scenarios are enumerated by m4 preprocessing, which is partially supported by ProVerif [5], but not by the IDE [16]. The work exposes vulnerabilities in the Italian electronic citizen card system.

In the context of post-compromise security, Blazy et al. [6] combine a hierarchy of attackers in terms of “(power, reach, access)” and quantify the time until a message exchange protocol (e.g. the Signal protocol) recovers from a compromise. Boureau et al. [7] continue this line with the help of ProVerif and Tamarin. Our paper aims to generalize aspects of these lines of work by focusing

16 B. Bisping et al.

on how to make it convenient to adapt and interpret such automatic surveys of threat scenarios.

It remains a problem for engineering applications that the applied π -calculus underpinning ProVerif (similarly for Tamarin) departs from classical programming languages. This kind of accessibility problem is addressed by tools such as Cryspen's Hax [1], which translates from Rust code to ProVerif models. It would also be interesting to transfer threat models between worlds like this.

Security benchmarking. Our work addresses how to compare the security of communicating systems, a natural question in the software engineering community. For example, Mendes et al. [15] discuss ways of choosing the most secure version among functionally equivalent implementations in the context of web services, and Oliveira et al. [18] turn to a trustworthiness score metric. We bring similar reasoning to the symbolic verification context.

Quantitative attack trees. In model-based security analysis, it is common to learn more about the feasibility of implementing attacks by deriving a quantitative conclusion from symbolic structures. For example, Schneier [20] already augments attack trees with prices such that the total price of the attack is computed from the prices for relevant branches, taking into account the type of the tree (e.g., AND or OR attack structure). Kumar et al. [11] perform a more refined quantitative analysis of attack trees to find Pareto fronts, generalized to a logic for expressing various metrics on attack trees by Nicoletti et al. [17]. In our work, we derive priced attack trees based on the automated analysis of attack scenarios with a simple price model.

7 Conclusion

In this paper, we have presented how to quantify the budgets that an attacker needs to break security properties, using symbolic verification. Our key contribution is the structured augmentation of protocol models with models of priced attacker capabilities applied to the ProVerif ecosystem. This leads to an exhaustive analysis of attacks that the adversary may perform. Nevertheless, the approach is limited by the classical problem of software testing that the attack model might have blind spots. Human insight is still needed on this front.

Regarding possible directions of future work, our approach is currently limited to pricing the presence of attack capabilities in a scenario. For a more refined analysis of how often an attacker uses their abilities, a richer preprocessing or a resource-aware derivation of attacks would be necessary, possibly going beyond ProVerif. Similar limitations exist with respect to more specific timing constraints of attacks and defenses, which play a role in the context of moving target defense strategies [13]. For both topics, as well as for the limitation of exponential blow-up of scenarios, it would also be interesting to have security solvers that address questions of attack prices and mitigations as built-in features.

References

1. Bhargavan, K., Buyse, M., Franceschino, L., Hansen, L.L., Kiefer, F., Schneider-Bensch, J., Spitters, B.: hax: Verifying security-critical Rust software using multiple provers. In: Protzenko, J., Raad, A. (eds.) *Verified Software. Theories, Tools and Experiments*. pp. 96–119. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-86695-1_7
2. Bispin, B., Schlehuber-Caissier, P., Matei, N.: CompareVerif: ProVerif batch analysis for protocol comparison (2026), <https://github.com/benkeks/compareverif>, GitHub Repository
3. Blanchet, B.: Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security* **1**(1–2), 1–135 (2016). <https://doi.org/10.1561/33000000004>
4. Blanchet, B.: The security protocol verifier proverif and its horn clause resolution algorithm. *Electronic Proceedings in Theoretical Computer Science* **373**, 14–22 (2022). <https://doi.org/https://doi.org/10.4204/eptcs.373.2>
5. Blanchet, B., Smyth, B., Cheval, V., Sylvestre, M.: Proverif 2.05: Automatic cryptographic protocol verifier, user manual and tutorial. <https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf> (2025)
6. Blazy, O., Boureanu, I., Lafourcade, P., Onete, C., Robert, L.: How fast do you heal? A taxonomy for post-compromise security in secure-channel establishment. In: *Proceedings of the 32nd USENIX Conference on Security Symposium. SEC '23*, USENIX Association, USA (2023). <https://doi.org/10.5555/3620237.3620568>
7. Boureanu, I., Onete, C., Wesemeyer, S., Robert, L., Miller, R., Lafourcade, P., Rajaona, F.: Post-compromise security with application-level key-controls – with a comprehensive study of the 5G AKMA protocol. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. p. 231–247. ASIA CCS '25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3708821.3733910>
8. David, S., Plesa, M., Irimia, S., Poenaru, R.: Privacy-preserving service for secure storage of passwords based on singularization. In: *2025 25th International Conference on Control Systems and Computer Science (CSCS)*. pp. 558–562 (2025). <https://doi.org/10.1109/CSCS66924.2025.00088>
9. Gaber, C., Macariot-Rat, G., David, S., Wary, J.P., Cuaboz, A.: Position paper: Strengthening applets on legacy SIM cards with singularization, a new moving target defense strategy. In: Bouzeffrane, S., Banerjee, S., Mourlin, F., Boumerdassi, S., Renault, É. (eds.) *Mobile, Secure, and Programmable Networking*. pp. 71–74. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-52426-4_5
10. Jacomme, C., Kremer, S.: An extensive formal analysis of multi-factor authentication protocols. *ACM Trans. Priv. Secur.* **24**(2) (Jan 2021). <https://doi.org/10.1145/3440712>
11. Kumar, R., Ruijters, E., Stoelinga, M.: Quantitative attack tree analysis via priced timed automata. In: Sankaranarayanan, S., Vicario, E. (eds.) *Formal Modeling and Analysis of Timed Systems - 13th International Conference, FORMATS 2015, Madrid, Spain, September 2-4, 2015, Proceedings. Lecture Notes in Computer Science*, vol. 9268, pp. 156–171. Springer (2015). https://doi.org/10.1007/978-3-319-22975-1_11
12. Lowe, G.: An attack on the Needham–Schroeder public-key authentication protocol. *Information Processing Letters* **56**(3), 131–133 (1995). [https://doi.org/10.1016/0020-0190\(95\)00144-2](https://doi.org/10.1016/0020-0190(95)00144-2)

18 B. Bisping et al.

13. Matei, N., Bisping, B., Schlehuber-Caissier, P., Kushik, N., Gaber, C., Wary, J.P.: Orange-Télécom SudParis collaboration: Formal verification of security properties in moving target defense strategies. In: *Approches Formelles pour l'Assistance au Développement de Logiciels (AFADL)* (2026)
14. Meier, S., Schmidt, B., Cremers, C., Basin, D.: The TAMARIN prover for the symbolic analysis of security protocols. In: Sharygina, N., Veith, H. (eds.) *Computer Aided Verification*. pp. 696–701. Springer Berlin Heidelberg, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_48
15. Mendes, N., Madeira, H., Duraes, J.: Security benchmarks for web serving systems. In: *2014 IEEE 25th International Symposium on Software Reliability Engineering*. pp. 1–12 (2014). <https://doi.org/10.1109/ISSRE.2014.38>
16. Moser, F.: VSCode ProVerif language service (2026), <https://github.com/ProVerif/vscode-proverif-language-service/>, GitHub Repository
17. Nicoletti, S.M., Lopuhaä-Zwakenberg, M., Hahn, E.M., Stoelinga, M.: ATM: A logic for quantitative security properties on attack trees. In: Ferreira, C., Willemse, T.A.C. (eds.) *Software Engineering and Formal Methods*. pp. 205–225. Springer Nature Switzerland, Cham (2023). <https://doi.org/10.1007/s10270-025-01323-z>
18. Oliveira, R.A., Raga, M.M., Laranjeiro, N., Vieira, M.: An approach for benchmarking the security of web service frameworks. *Future Generation Computer Systems* **110**, 833–848 (2020). <https://doi.org/10.1016/j.future.2019.10.027>
19. Paier, M., Van Eeden, R., Miculan, M.: Formal analysis of multi-factor authentication schemes in digital identity cards. In: Madeira, A., Knapp, A. (eds.) *Software Engineering and Formal Methods*. pp. 423–440. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-77382-2_24
20. Schneier, B.: Attack trees. <https://www.schneier.com/wp-content/uploads/2015/12/attacktrees.pdf> (1999)